

FitSpec

A Universal GUI Spectral Fitting Framework in Python

Project Architecture README

August 17, 2026

Contents

1	Project Overview	6
2	High-Level Architecture	6
3	Common Core	7
3.1	Universal Spectrum Object	7
3.2	Input/Output	8
3.3	Air/Vacuum Wavelengths	8
3.4	Explicit Component Parameters	8
3.5	Generic Models and Fitting	9
3.6	Statistics	9
3.7	Configuration	9
3.8	Common Utilities	10
4	Science Modules	10
5	Unified Stellar-Population Fitting	10
5.1	Motivation and design philosophy	10
5.2	Common stellar-library representation	11
5.3	Library-specific normalization and metadata	12
5.4	Automatic UV/optical classification without separate fitting engines	12
5.5	Observed spectrum and fitting mask	12
5.6	Flux scaling	13
5.7	Model transformation	13
5.7.1	Logarithmic wavelength representation	13
5.7.2	Intrinsic stellar velocity dispersion	14
5.7.3	Instrumental resolution	14
5.7.4	Residual stellar velocity	14
5.7.5	Internal attenuation	14
5.8	Separable nonlinear least squares and variable projection	15
5.9	Deterministic SSP-basis selection	16
5.10	Optional nuisance gas templates	16
5.11	Outer nonlinear minimization	16
5.12	Degrees of freedom and goodness of fit	16

5.13	Local parameter uncertainties	17
5.14	Stellar-population diagnostics	17
5.14.1	SSP correlation matrix	17
5.14.2	Single-SSP χ^2 landscape	17
5.14.3	Singular values, effective rank, and condition number	17
5.14.4	Dominant-SSP degeneracies	18
5.15	Mass and light fractions	18
5.16	Bayesian stellar-population inference	18
5.17	Saved stellar-fit and plotting products	19
5.18	What is new in the FitSpec stellar approach	20
5.19	Emission-Line Fitting	20
5.19.1	Overview and relationship to the rest of FitSpec	20
5.19.2	The nebular line list and its atomic provenance	21
5.19.3	Explicit-component kinematic architecture	22
5.19.4	Line-profile model	23
5.19.5	Line-list selection	24
5.19.6	User-specified fixed amplitude ratios	24
5.19.7	Flux pre-scaling	25
5.19.8	Automatic component-count selection	25
5.19.9	Fitting engine and results	25
5.20	Posterior inference integration	25
5.21	Absorption-Line Fitting	27
5.21.1	Overview	27
5.21.2	Atomic transition data	27
5.21.3	Explicit-component kinematic architecture	28
5.21.4	Optical-depth model	28
5.21.5	Partial covering	29
5.21.6	Fitting engine and results	29
5.21.7	Posterior inference for single-group absorption fits	30
5.22	Advanced Absorption Features	30
5.22.1	A generic parameter-tying primitive	31
5.22.2	Joint, multi-system (cross-ion) fitting	31
5.22.3	Thermal and turbulent Doppler-parameter linking	31
5.22.4	Fixed-ratio and common-pattern column-density ties	32
5.22.5	Subpixel profile oversampling	32
5.22.6	Region velocity shifts and in-fit continuum adjustment	32
5.22.7	The unidentified-line marker	32
5.22.8	Automatic rejection of insignificant components	32
5.22.9	Column-density upper limits	33
5.22.10	Synthetic spectrum generation	34
5.22.11	Posterior inference for joint, tied, and rejected absorption models	34
6	Continuum Estimation	35
6.1	Motivation	35
6.2	The FitSpec custom continuum algorithm	35
6.2.1	Step 1: finite-pixel selection	35
6.2.2	Step 2: preliminary boxcar smoothing	35
6.2.3	Step 3: local noise-scale estimate	36

6.2.4	Step 4: local maxima/minima and envelope construction	36
6.2.5	Step 5: adaptive feature-detection thresholds	36
6.2.6	Step 6: expand the feature masks	36
6.2.7	Step 7: derivative-based continuum-point weights	37
6.2.8	Step 8: percentile selection	37
6.2.9	Step 9: Savitzky–Golay smoothing	37
6.2.10	Step 10: continuous continuum interpolation	37
6.2.11	Step 11: continuum-scatter diagnostic	38
6.3	Why the custom algorithm is different from a simple polynomial	38
6.4	Alternative continuum estimators	38
6.4.1	Polynomial continuum	38
6.4.2	Cubic interpolation mode	38
6.4.3	LOWESS	38
6.4.4	Local Gaussian continuum	39
6.4.5	Peak/trough interpolation	39
6.4.6	Median filtering	39
6.4.7	Friends-of-friends / DBSCAN mode	39
6.4.8	PCA mode	39
6.5	Interaction with masking	40
6.6	Interaction with rebinning and smoothing	40
6.7	Manual continuum interaction	40
6.8	Continuum uncertainty and future development	40
6.9	What is new in the FitSpec continuum approach	40
7	Generic Inference Framework	41
7.1	Implemented package structure	41
7.2	Deterministic inference	42
7.3	Posterior problem abstraction	42
7.4	Likelihood and covariance handling	43
7.5	Priors	43
7.6	emcee posterior sampling	43
7.7	dynesty nested sampling	44
7.8	Posterior result products	44
7.9	Posterior diagnostics	45
7.10	Model selection	45
7.11	Relationship to the science modules	45
7.12	Recommended terminal workflow	46
8	GUI Architecture	47
8.1	Explicit Component Controller	48
8.2	Implementation	48
8.2.1	Common application shell and shared session state	49
8.2.2	Mask controller and the three-tier mask model	49
8.2.3	Explicit component controller	50
8.2.4	Stellar, emission, and absorption panels	51

9	Rebinning, Smoothing, Resolution, and Masking	51
9.1	Permanent Rebinning Versus Display Smoothing	51
9.2	Spectral Resolution and Instrumental Convolution	52
9.3	Masking	52
10	Results, Saving, and Plotting	53
10.1	Plotting	54
10.1.1	Architecture	54
10.1.2	Generic primitives	54
10.1.3	Emission-fit plotting	55
10.1.4	Absorption-fit plotting	55
10.1.5	Stellar-fit plotting	55
11	FitSpec Test Run	55
12	Directory Structure	56
13	Design Principles	58
14	Stellar Population Libraries	59
14.1	Purpose and scope	59
14.2	Common HDF5 design	60
14.3	BPASS v2.2.1	60
14.3.1	Source release and motivation	60
14.3.2	Population-spectrum products	61
14.3.3	Individual BPASS stellar models	61
14.3.4	WM-Basic atmosphere archive	61
14.3.5	Atmosphere normalization and reconstruction	62
14.3.6	Population-library product	62
14.4	Starburst99 / pySB99	63
14.4.1	Scientific basis	63
14.4.2	pySB99 population-library construction	63
14.4.3	Nebular continuum policy	64
14.5	SLUG2	64
14.5.1	Why SLUG was added	64
14.5.2	Compilation and runtime environment	64
14.5.3	Track families discovered in the local SLUG installation	64
14.5.4	Spectral synthesis mode and atmosphere assignment	65
14.5.5	Nebular processing must be disabled explicitly	65
14.5.6	Deterministic versus stochastic mode	65
14.5.7	The PARSEC interpolation problem	66
14.5.8	Completed SLUG HDF5 library	66
14.5.9	SLUG wavelength coverage and scientific use	67
14.6	e-MILES	67
14.6.1	Why e-MILES is the optical backbone	67
14.6.2	Repository acquisition and configuration	68
14.6.3	Native repository structure	68
14.6.4	Wavelength grid	69

14.6.5 Padova00 grid	69
14.6.6 BaSTI grid	69
14.6.7 Fitting-oriented e-MILES HDF5	69
14.7 Cross-library roles	70
14.8 Nebular continuum policy across libraries	71
14.9 Resolution and sampling policy	71
14.10 Lazy access and compartmentalization	71
14.11 Validation checklist	72
14.12 Builder scripts and permanent products	72
14.13 External documentation and web resources used	72
14.14 References that should accompany use of each library	73

1 Project Overview

FitSpec is intended to be a single, universal spectral-fitting application written entirely in Python. The user should interact with one program, while the internal implementation remains modular and separates common infrastructure from stellar, emission-line, and absorption-line science modules.

The three principal fitting modes are:

1. stellar-population and stellar-continuum fitting;
2. emission-line fitting;
3. absorption-line fitting.

The central design objective is to avoid maintaining three largely independent GUI programs. Shared operations such as loading spectra, masking, rebinning, spectral-resolution handling, model convolution, optimization, saving, plotting, and GUI state management should be implemented once and reused by all fitting modes.

The application now has one primary executable entry point:

```
python fitspec.py spectrum.dat
```

or, when packaged as a command-line application and a local `config.dat` supplies `input_spectrum`, `redshift`, and optionally `mode`,

```
fitspec
```

Explicit command-line values override the corresponding configuration entries, so the same executable supports both reproducible run-directory configurations and temporary per-launch overrides.

This does *not* imply placing all implementation code in one very large Python file. FitSpec should appear as one program to the user while remaining a structured Python package internally.

2 High-Level Architecture

FitSpec is organized into four conceptual layers:

1. **Core infrastructure:** spectrum representation, I/O, configuration, rebinning, resolution, masking, parameters, generic numerical utilities, statistics, and common result objects.
2. **Science modules:** stellar, emission-line, absorption-line, and continuum-specific functionality.
3. **Inference:** reusable deterministic minimization, likelihood evaluation, model selection, priors, and posterior-sampling infrastructure.
4. **Interfaces and output:** GUI, command-line execution, plotting, saving, loading, and diagnostics.

The important conceptual separation is between the **physical model**, the **science-specific fitting strategy**, and the **generic inference utilities**. FitSpec should reuse optimization, likelihood, model-selection, and posterior-sampling machinery whenever the mathematical problem is genuinely shared, but it should not force all science modules into an artificial common optimizer interface.

This distinction is especially important for stellar fitting. The stellar module uses the unified UV/optical variable-projection architecture developed for FitSpec: at each nonlinear trial, common

stellar parameters are applied to the full SSP basis and the non-negative SSP coefficients are solved as a bounded linear problem. This specialized deterministic structure should be retained. Generic inference utilities may be called by the stellar module where appropriate, but the stellar fitter does not need to be rewritten merely to conform to a generic wrapper.

For fitting problems that naturally follow a standard parameter-to-model mapping, a generic problem description may still be used. Conceptually:

```
problem = FitProblem(
    wave=wave[mask],
    flux=flux[mask],
    flux_unc=flux_unc[mask],
    model=model,
    parameters=parameters,
)

result = deterministic.fit(problem, method="least_squares")
```

The same saved model state may later be evaluated by model-selection or posterior-sampling tools. Science modules therefore expose enough information for inference to operate reproducibly, while remaining free to use specialized deterministic algorithms when those algorithms are intrinsic to the fitting problem.

3 Common Core

3.1 Universal Spectrum Object

A central `Spectrum` object should be the common data container used by every science module. Rather than repeatedly passing wavelength, flux, uncertainty, continuum, fit, and mask arrays separately, the object should hold them together and validate dimensional consistency once.

A possible conceptual definition is:

```
@dataclass
class Spectrum:
    wave: np.ndarray
    flux: np.ndarray
    flux_unc: np.ndarray | None = None
    continuum: np.ndarray | None = None
    model: np.ndarray | None = None
    mask: np.ndarray | None = None
    redshift: float = 0.0
    resolution: object = None # ResolutionModel, once core.resolution is imported
    metadata: dict = field(default_factory=dict)
```

Missing optional information should remain explicitly absent rather than causing input failure. For example, a two-column wavelength–flux spectrum should be valid even when uncertainty, continuum, or previously fitted models are unavailable.

`Spectrum.from_arrays(...)` is the standard constructor: it runs structural cleaning (finite, sorted, unique wavelengths; see `core.spectrum.clean_spectrum`) and initializes `mask` from the resulting invalid-pixel flags via `core.masking.combine_masks`, so a freshly constructed `Spectrum` already excludes non-finite flux/uncertainty before any region, velocity, or interactive masking is layered on. Non-finite flux or non-finite/non-positive uncertainty are flagged in this initial mask rather than removed from the arrays – every array stays pixel-aligned with the original input.

3.2 Input/Output

The I/O layer (`core.io.load_spectrum`) converts supported file types into the same internal `Spectrum` representation, dispatching on file extension. Two families of input are supported:

- **Delimited text** (comma, tab, or whitespace, auto-detected): a header row names columns (matched case-insensitively against known aliases, e.g. `wavelength/lambda/wave`, `flux_err/error/sigma`); a headerless file uses the positional order `wave`, `flux`, `flux_unc`, `continuum`, `model`, with any trailing columns simply omitted.
- **FITS binary tables**, matched against the same column-name aliases. Multi-row array-valued columns (e.g. one row per spectral segment, as in HST/COS x1d-style products) are flattened across rows before cleaning, so overlapping or out-of-order segments are sorted and deduplicated by `clean_spectrum` rather than needing special-cased handling.

An unrecognized wave/flux column raises immediately rather than guessing. FITS images/cubes (a WCS-based wavelength axis with spatial extraction) are not yet supported – a substantially different loading path left for later.

Wavelength-medium (air/vacuum) conversion and permanent rebinning are deliberately not performed inside the loader; both are config-driven decisions applied by the caller immediately afterward (see the Air/Vacuum Wavelengths and Rebinning subsections).

3.3 Air/Vacuum Wavelengths

FitSpec’s internal wavelength standard is vacuum: the stellar libraries (SB99, BPASS, eMILES) are natively vacuum, and vacuum wavelength is the physically native quantity for redshift/Doppler work. `core.wavelengths` converts anything expressed in air wavelengths – a line list, or an observed spectrum whose reduction pipeline delivered air-calibrated wavelengths – to vacuum before it enters any fitting code, using the standard dispersion-of-air formula (Morton 1991, ApJS 77, 119; the same one used by the IDL Astronomy User’s Library’s `airtovac/vactoir` and consistent with the NIST Atomic Spectra Database).

The standard atomic-spectroscopy convention – wavelengths below 2000 Å are, by convention, already vacuum; wavelengths at or above 2000 Å are, by convention, air unless otherwise stated – is implemented as `standard_medium(wave)` and used via `medium="auto"`. This is appropriate for a line list built to that convention (verified against the project’s nebular emission-line list, which follows it exactly with no exceptions found), but is *not* appropriate for an observed spectrum’s wavelength axis, which is uniformly one medium regardless of wavelength – use an explicit `medium="air"` or `medium="vacuum"` for that. Absorption-line reference compilations built on Morton’s own catalogs (e.g. a VPFIT-style `atom.dat`) are typically vacuum throughout, with no split at 2000 Å – verified against the project’s absorption line list – so those should be loaded with an explicit `medium="vacuum"`, not `"auto"`.

3.4 Explicit Component Parameters

`core.parameters` implements the "explicit state beats implicit inference" design principle directly. A `Parameter` is one named scalar (value, bounds, a `fixed` flag); a `Component` is an ordered set of named `Parameters` (e.g. amplitude, velocity, velocity dispersion for one emission kinematic component); a `ModelParameters` holds an explicit `n_components: int` kept in lockstep with its component list (enforced at construction and by every mutating method, so it can never silently drift), plus an `active_component_index`.

Changing which component is active does not modify any component's values – every component retains its own settings independently, which is what lets the GUI's active-component slider work the way the fundamentals doc describes: switching the active component changes which one the amplitude/velocity/velocity-dispersion widgets edit, while every other component keeps its last-set values in memory. `to_vector/from_vector/bounds` pack and unpack the free (non-fixed) parameters for direct use with an optimizer or sampler; `to_dict/from_dict` support the save/load button.

3.5 Generic Models and Fitting

`core.models` holds only numerical primitives shared by every science module – a peak-normalized `gaussian` and a Faddeeva-function `voigt_profile` – plus `sum_components`, a driver that sums whatever components a `ModelParameters` holds using an injected per-component model function, so the driver itself never needs to know whether it is summing emission-line Gaussians or absorption Voigt profiles. Species-specific models (which lines, which library) stay in the science modules.

`core.fitting.fit_deterministic` is the generic bounded nonlinear least-squares driver requested in the fundamentals doc: given `wave`, `flux`, `flux_unc`, a `mask`, a `ModelParameters`, and a `model_func(wave, model_parameters) -> flux`, it fits the free parameters via `scipy.optimize.curve_fit` and returns a `FitResult`. The optimizer itself is completely agnostic to what `model_func` does internally (profile shapes, resolution convolution, redshift) – that separation is what lets stellar, emission, and absorption fitting share one deterministic engine.

3.6 Statistics

`core.statistics` is the single implementation of chi-square, reduced chi-square, and information criteria, so every science module's fit statistics are directly comparable rather than each maintaining its own copy of the math. `compute_fit_statistics(...)` is the one-call entry point, bundling:

- chi-square and reduced chi-square;
- an **effective sample size** (`n_data / pix_per_resel`), correcting BIC's $\ln(N)$ penalty for oversampled/correlated pixels;
- a maximum-likelihood **error-jitter scale** s (with $\sigma_{\text{eff}} = s\sigma$), letting quoted uncertainties self-calibrate against the actual residual scatter rather than penalizing additional model components simply because quoted uncertainties were too small; floored at 1.0 by default so uncertainties are only ever inflated, never shrunk below their quoted values;
- the full Gaussian $-2 \ln L$ (required, not just χ^2 , for BIC/AIC to be meaningful once jitter is involved);
- BIC, AIC, and AICc (the small-sample-corrected AIC, relevant whenever a component search's sample size is not much larger than its parameter count).

3.7 Configuration

FitSpec parameters may be altered without editing any Python script, through three layers with a strict, fixed precedence (lowest to highest):

1. **Base default config** (`config/default_config.dat`): defaults shared across all three fitting modes.

2. **Mode-specific default config** (`config/default_config_{stellar,emission,absorption}.dat`): defaults for one fitting mode. A keyword here overrides the same keyword in the base default config.
3. **User config** (`config.dat`): overrides any default for the current run.
4. **Command-line arguments**: override both default and user config for the current run.

Together, levels 1 and 2 define the complete, authoritative set of recognized configuration keywords – every keyword FitSpec will ever read must have a default value declared in one of these four files. This set doubles as a schema: each keyword’s default value fixes its expected type, and `config.dat`/CLI values are checked and coerced against it.

Unknown keywords are a fatal error. If `config.dat` or a command-line argument names a keyword absent from every default config file, FitSpec must raise an error immediately rather than warn or silently accept it. This is the configuration-layer instance of the same fail-fast philosophy as the resolution warning: a typo’d keyword that is silently ignored produces a confusing wrong-default bug far from its cause.

Command-line arguments are generated automatically from whatever keywords exist in the default config files, rather than maintained as a separate, hand-curated argument list. This guarantees every configurable value is reachable from the command line and that the default config files remain the single source of truth for what is configurable at all.

3.8 Common Utilities

Reusable calculations should be defined once. Examples include:

- nearest-value lookup;
- wavelength/velocity conversion;
- velocity-to-redshift shift conventions;
- Gaussian width conversion between km s^{-1} and Angstrom;
- cleaning finite spectral pixels (`core.spectrum.clean_spectrum`);
- flux-conservative rebinning (`core.rebinning.rebin_spectrum`);
- instrumental-resolution lookup and application (`core.resolution`, see Section 9);
- model resampling to the observed wavelength grid.

4 Science Modules

5 Unified Stellar-Population Fitting

5.1 Motivation and design philosophy

The stellar-population module in FitSpec is intentionally not a wrapper around a collection of independent ultraviolet and optical fitting programs. Instead, FitSpec introduces a single, wavelength-agnostic stellar fitting architecture in which the same mathematical optimization problem is used for rest-frame UV and optical spectra. The wavelength regime determines which stellar libraries

and configuration defaults are appropriate, but it does not select a different fitting backend. This is a central design choice of FitSpec.

The motivation is both scientific and computational. Existing stellar-population synthesis packages provide spectra with very different native age grids, metallicity grids, wavelength coverage, spectral resolution, normalization conventions, and assumptions about stellar evolution. Examples include Starburst99 (Leitherer et al., 1999a), BPASS (Eldridge et al., 2017a; Stanway & Eldridge, 2018a), SLUG (Krumholz et al., 2015a), and E-MILES (Vazdekis et al., 2016a). Historically, these differences often lead fitting codes to be written around one particular library or wavelength range. FitSpec instead separates the *stellar-population library* from the *fitting algorithm*. Each supported stellar library is translated into a common internal representation, after which the same transformation and fitting machinery is applied.

The current FitSpec implementation uses HDF5 population libraries for Starburst99/pySTARBURST99, BPASS, SLUG, and E-MILES. The original distribution formats are converted beforehand into a FitSpec-readable HDF5 schema. Consequently, the fitting code does not need to understand the native file organization of each synthesis package during a science run. Only the subset of ages, metallicities, wavelengths, and model branches required for the current spectrum is loaded into memory.

5.2 Common stellar-library representation

Each selected single stellar population (SSP) is represented internally by a spectrum

$$S_j(\lambda), \tag{1}$$

where the index j uniquely identifies an age–metallicity combination and any additional library-specific branch information such as the IMF, rotation prescription, binary/single-star population, or atmosphere library.

For every SSP, FitSpec retains explicit metadata including

- SSP age;
- absolute or relative metallicity;
- library-specific metallicity label;
- reference or formed stellar mass represented by the model;
- surviving stellar-mass fraction when supplied by the library;
- intrinsic spectral-resolution metadata when it is explicitly known;
- the model family and branch used to construct the spectrum.

The spectra are converted to a common luminosity-density-per-unit-mass representation whenever the native model normalization permits this. The fitting coefficients can therefore retain a direct physical interpretation as stellar-mass scales rather than arbitrary template weights. This is particularly important for comparing young UV-bright populations with older optical populations.

The HDF5 layer is deliberately independent of the optimizer. Adding a future stellar library should therefore require a new loader that returns the common FitSpec stellar-library object, not a new fitting algorithm.

5.3 Library-specific normalization and metadata

A central requirement of the common stellar interface is that library-specific file conventions are resolved in the loader, not in the optimizer and not by target-specific configuration guesses. The returned SSP spectra are therefore placed on a common luminosity-density-per-unit-mass scale whenever the HDF5 metadata support a physical normalization.

For BPASS, the HDF5 reference mass and branch metadata are used directly to construct spectra per unit stellar mass. For Starburst99/pySTARBURST99, FitSpec determines the physical flux scaling from the HDF5 metadata and provenance automatically; `sb99_flux_unit` is not a required science configuration parameter. In the current pySTARBURST99 HDF5 schema, the stored native linear high-resolution spectrum retains the historical synthesis scaling of 10^{-20} relative to physical luminosity density. FitSpec therefore applies the corresponding factor of 10^{20} before converting from $\text{erg s}^{-1} \text{ \AA}^{-1}$ to $L_{\odot} \text{ \AA}^{-1}$ and dividing by the population reference mass. Future HDF5 libraries may instead provide an explicit physical scale attribute, which takes precedence. If neither explicit physical metadata nor recognized pySTARBURST99 provenance is available, FitSpec raises an error rather than guessing a flux unit and returning a meaningless stellar mass.

For SLUG, the selected deterministic HDF5 branch and its metallicity/reference-mass metadata are retained in the common library object. All stellar-library families expose a common `library_metadata` interface so that branch identity can be recorded in terminal output, saved products, diagnostics, and posterior provenance.

5.4 Automatic UV/optical classification without separate fitting engines

FitSpec classifies the rest-frame spectral coverage before choosing the default stellar library. A configurable boundary wavelength is used to identify UV-dominated and optical-dominated spectra. The classification controls default library selection and sensible age/metallicity ranges, but **does not dispatch the fit to a different algorithm**.

The current default behavior is:

- UV coverage: Starburst99/pySTARBURST99 by default, with BPASS and SLUG available;
- optical coverage: E-MILES by default;
- explicit user selection: any supported library may be requested if its wavelength coverage overlaps the data.

For UV libraries, branch selection is library-specific but resolved before the common fitter is called. BPASS may be restricted by IMF, binary/single-star population, metallicity codes, and age grid. Starburst99/pySTARBURST99 may be restricted by evolutionary track, rotation branch, atmosphere/spectral-synthesis library, metallicity label, and age. SLUG deterministic libraries may be selected explicitly by family/subfamily/spectral-synthesis branch, or the corresponding settings may be set to `auto`; in that case FitSpec discovers the deterministic branches actually present in the HDF5 file, prints the selected branch, and reports the available branches if an explicitly requested path does not exist.

This allows FitSpec to preserve the physical advantages of specialized stellar libraries while avoiding duplicated UV and optical fitting codes.

5.5 Observed spectrum and fitting mask

The stellar fitter operates on the FitSpec `Spectrum` object after all permanent preprocessing has been completed. In particular, permanent rebinning, if explicitly requested, has already occurred

before stellar fitting begins. GUI smoothing remains display-only and never changes the fitted spectrum.

The stellar fitting wavelength is the rest-frame wavelength

$$\lambda_{\text{rest}} = \frac{\lambda_{\text{obs}}}{1+z}. \quad (2)$$

Only pixels included in the current FitSpec working mask participate in the objective function. Stellar fitting does not use line-centred velocity-window masks because no discrete transition list defines the stellar fit. Instead, the stellar mask is established through included/excluded wavelength intervals and interactive point or rectangular masking.

For diagonal flux uncertainties, the weighted residual at pixel i is

$$r_i = \frac{F_i - M_i}{\sigma_i}, \quad (3)$$

and the deterministic objective is

$$\chi^2 = \sum_{i \in \mathcal{M}} r_i^2, \quad (4)$$

where $\mathcal{M}$ denotes the active stellar-fitting mask.

5.6 Flux scaling

For a source with luminosity distance D_L , the observed F_λ spectrum is converted to a rest-frame luminosity-density scale according to

$$L_\lambda = 4\pi D_L^2 (1+z) F_\lambda. \quad (5)$$

FitSpec expresses the result in $L_\odot \text{ \AA}^{-1}$ when the input and stellar-library normalization support this physical scaling. Solar luminosity and other physical constants are taken from `astropy.constants`.

For $z = 0$ objects, the current implementation retains the historical 10 kpc fallback used by the standalone fitter unless a physical distance is supplied explicitly. This fallback is maintained for reproducibility but should not be interpreted as a measured distance.

5.7 Model transformation

At every nonlinear trial, the selected SSP basis is transformed using one shared stellar attenuation and fixed or fitted stellar kinematics. The available nonlinear stellar parameters are

$$\theta_\star = [E(B-V), v_\star, \sigma_\star], \quad (6)$$

but they are not all mandatory fit dimensions. For the lightweight default stellar run, `stellar_fit_velocity=False` and `stellar_fit_sigma=False`, so $v_\star$ and $\sigma_\star$ are held at their configured values unless the user explicitly enables either parameter. This preserves fast first-pass behavior while still allowing velocity and dispersion to be fitted independently when required.

The transformation applied to each SSP includes the following operations.

5.7.1 Logarithmic wavelength representation

Velocity shifts and velocity broadening are naturally multiplicative in wavelength. FitSpec therefore constructs an internal uniformly sampled $\ln \lambda$ grid for model convolution. On this grid, a Gaussian velocity dispersion corresponds approximately to a wavelength-independent Gaussian width in pixels.

5.7.2 Intrinsic stellar velocity dispersion

A Gaussian stellar line-of-sight velocity dispersion is applied to every SSP using the common configured σ_* ; it is optimized only when stellar-sigma fitting is explicitly enabled. If the instrumental and intrinsic template line-spread functions are approximately Gaussian, variances add under convolution. This allows the required model broadening to be constructed from the stellar dispersion and the instrumental resolution.

5.7.3 Instrumental resolution

Every model evaluated against data must account for the observational line-spread function. FitSpec does not equate model wavelength sampling with intrinsic spectral resolution. An intrinsic template resolution is used only when it is explicitly known from the model library or supplied by the user.

For Gaussian broadening, the additional instrumental broadening required to match the observation follows

$$\sigma_{\text{conv}}^2(\lambda) = \sigma_{\text{obs}}^2(\lambda) - \sigma_{\text{model}}^2(\lambda), \quad (7)$$

when the observation is broader than the native template. If the requested observational resolution is finer than the template resolution, FitSpec cannot deconvolve information that is absent from the library and must issue a warning rather than silently sharpening the template.

The observational resolution may be wavelength-dependent. Consequently, FitSpec’s resolution layer permits variable Gaussian convolution rather than assuming that one resolving power describes the entire spectrum.

5.7.4 Residual stellar velocity

The stellar velocity, whether fixed or fitted, uses the relativistic Doppler factor

$$\mathcal{D} = \sqrt{\frac{1 + \beta}{1 - \beta}}, \quad \beta = \frac{v_*}{c}, \quad (8)$$

so that

$$\lambda' = \mathcal{D}\lambda. \quad (9)$$

The shifted model is then interpolated onto the wavelength grid used by the observation.

The interpolation/convolution path explicitly avoids interpreting an unresolved broadening kernel as zero physical flux. When the requested Gaussian kernel is narrower than the native template sampling, the transformation approaches the interpolation limit rather than leaving output pixels at an initialized zero value. Wavelengths genuinely outside template support are treated as unsupported/masked rather than as zero stellar continuum. This prevents artificial narrow “holes” in otherwise positive UV SSP spectra.

5.7.5 Internal attenuation

FitSpec applies one stellar attenuation curve to the composite SSP basis. The default is the Calzetti attenuation relation for star-forming systems (Calzetti et al., 2000), while alternative attenuation/extinction prescriptions may be selected through configuration.

For an attenuation curve $k(\lambda)$,

$$T(\lambda) = 10^{-0.4 E(B-V) k(\lambda)}, \quad (10)$$

and each transformed SSP becomes

$$S'_j(\lambda; \boldsymbol{\theta}_\star) = S_j(\lambda) T(\lambda) \quad (11)$$

after the appropriate kinematic and instrumental transformations.

5.8 Separable nonlinear least squares and variable projection

The central mathematical feature of the FitSpec stellar fitter is that stellar population weights enter the spectrum *linearly*, while attenuation and kinematics enter *nonlinearly*. Problems with this structure are known as separable nonlinear least-squares problems. Eliminating or solving the linear variables inside each nonlinear trial is the basis of the variable-projection family of methods (Golub & Pereyra, 1973; O’Leary & Rust, 2013).

For a transformed stellar basis, the stellar model is

$$M_\star(\lambda; \boldsymbol{\theta}_\star, \mathbf{m}) = \sum_{j=1}^{N_{\text{SSP}}} m_j S'_j(\lambda; \boldsymbol{\theta}_\star), \quad (12)$$

with

$$m_j \geq 0. \quad (13)$$

The non-negative condition prevents the optimizer from creating unphysical cancellation between positive and negative stellar populations. Non-negative and bounded least-squares methods have a long history in numerical optimization (Lawson & Hanson, 1995; Stark & Parker, 1995); FitSpec uses SciPy’s bounded linear least-squares infrastructure (Virtanen et al., 2020).

At every nonlinear trial:

1. construct the transformed SSP basis for the trial $\boldsymbol{\theta}_\star$;
2. restrict the basis and observed spectrum to the active fitting mask;
3. divide each fitted pixel by its uncertainty;
4. scale design-matrix columns to improve numerical conditioning;
5. solve the bounded linear least-squares problem for all SSP coefficients;
6. reconstruct the total model;
7. return the weighted residual vector to the outer nonlinear optimizer.

The outer problem therefore minimizes

$$\chi^2(\boldsymbol{\theta}_\star) = \min_{\mathbf{m} \geq 0} \sum_{i \in \mathcal{M}} \left[\frac{F_i - \sum_j m_j S'_{ij}(\boldsymbol{\theta}_\star)}{\sigma_i} \right]^2. \quad (14)$$

This is the key unification between UV and optical fitting in FitSpec: the same optimization architecture is used regardless of which stellar library supplies the SSP basis.

5.9 Deterministic SSP-basis selection

Large HDF5 libraries can contain hundreds or thousands of SSPs. Solving a bounded non-negative mixture over every template at every nonlinear trial can dominate the runtime even when only one nonlinear parameter is active. FitSpec therefore supports two deterministic basis modes.

candidate the default for large unrestricted libraries. FitSpec first evaluates a vectorized single-SSP scaling and χ^2 ranking over the full loaded library, retains the best `stellar_candidate_max` templates, and performs the repeated NNLS mixture solve only on that reduced basis. The returned coefficient arrays retain full-library indexing.

full solve the non-negative mixture using every SSP in the already filtered library. This is the recommended parity mode when the user has explicitly restricted the grid to a modest set, for example 4 BPASS metallicities $\times$ 10 ages = 40 SSPs.

Candidate screening is a computational approximation rather than a statement that excluded SSPs have zero scientific probability. Users testing convergence should compare candidate caps or use **full** mode for deliberately small grids.

5.10 Optional nuisance gas templates

Strong nebular emission can bias a stellar-continuum solution if those pixels are included in the stellar fit. FitSpec therefore permits optional Gaussian gas templates to be included *only as nuisance basis functions* during stellar fitting.

When enabled, the complete model is

$$M(\lambda) = \sum_j m_j S'_j(\lambda) + \sum_k a_k G_k(\lambda), \quad (15)$$

where G_k is a Gaussian template at a known nebular transition and $a_k \geq 0$ is its fitted amplitude. The gas templates may share a gas velocity and gas velocity dispersion, which enlarge the nonlinear parameter vector. Nuisance gas is disabled by default (`stellar_include_gas=False`) and must be explicitly enabled; it is not a substitute for the dedicated emission-line fit.

These nuisance components exist only to protect the stellar solution. Their amplitudes are not intended to replace the dedicated FitSpec emission-line module, which will provide the physically interpretable emission-line measurements.

5.11 Outer nonlinear minimization

The nonlinear residual vector is passed to a bounded nonlinear least-squares solver. The current implementation uses the Trust Region Reflective strategy available in SciPy's `least_squares` framework (Virtanen et al., 2020). Bounds are imposed on attenuation, stellar velocity, and stellar velocity dispersion, and, when nuisance gas fitting is active, on gas velocity and gas dispersion.

The nonlinear optimizer never treats every SSP coefficient as a nonlinear parameter. This is essential for tractability when hundreds of age-metallicity SSPs are available.

5.12 Degrees of freedom and goodness of fit

After convergence, FitSpec evaluates

$$\chi^2_\nu = \frac{\chi^2}{\nu}, \quad (16)$$

where the effective number of fitted parameters includes the active non-zero SSP coefficients, active nuisance gas components, and nonlinear fitted parameters. The exact bookkeeping is stored with the result so that the reported reduced χ^2 is reproducible.

5.13 Local parameter uncertainties

The deterministic nonlinear covariance is estimated from the optimizer Jacobian using a singular-value-decomposition pseudoinverse. Schematically,

$$\text{Cov}(\boldsymbol{\theta}) \simeq \left(J^\top J \right)^+, \quad (17)$$

optionally scaled by the final reduced χ^2 . The SVD treatment prevents formally singular directions from causing an immediate matrix-inversion failure.

These uncertainties are local Gaussian approximations. They do not replace posterior inference and should not be interpreted as a full characterization of multimodal age–metallicity–dust degeneracies.

5.14 Stellar-population diagnostics

A deterministic best fit is not sufficient for stellar-population interpretation because SSP spectra can be highly degenerate. FitSpec therefore calculates diagnostics describing the structure of the transformed SSP basis.

5.14.1 SSP correlation matrix

For every pair of transformed SSPs, FitSpec evaluates their spectral correlation over the fitted pixels. The resulting matrix identifies age–metallicity combinations that become nearly indistinguishable after attenuation, broadening, instrumental convolution, masking, and resampling.

5.14.2 Single-SSP χ^2 landscape

Every SSP is independently scaled against the observation and assigned a single-SSP χ^2 . The difference

$$\Delta\chi_j^2 = \chi_j^2 - \min_k \chi_k^2 \quad (18)$$

provides an intuitive age–metallicity landscape independent of the final multi-SSP mixture.

5.14.3 Singular values, effective rank, and condition number

The weighted design matrix is decomposed using singular-value decomposition,

$$A = U \Sigma V^\top. \quad (19)$$

The singular values reveal how many independent spectral directions are actually constrained by the selected wavelength coverage and mask. FitSpec records an effective rank and condition number to expose situations where the number of available SSP templates substantially exceeds the number of distinguishable spectral degrees of freedom.

5.14.4 Dominant-SSP degeneracies

The SSP contributing the largest fitted coefficient is identified as the dominant SSP. FitSpec reports the templates most strongly correlated with that component and can directly compare their transformed spectra. This makes it possible to determine whether a formally dominant age/metallicity solution is genuinely unique or merely one representative of a nearly degenerate family.

5.15 Mass and light fractions

Where the model normalization permits a physical mass interpretation, FitSpec computes relative stellar-mass fractions

$$f_{M,j} = \frac{m_j}{\sum_k m_k}. \quad (20)$$

It also estimates emergent light fractions over the fitted wavelength interval,

$$f_{L,j} = \frac{\int_{\lambda_1}^{\lambda_2} m_j S'_j(\lambda) d\lambda}{\sum_k \int_{\lambda_1}^{\lambda_2} m_k S'_k(\lambda) d\lambda}. \quad (21)$$

For libraries such as E-MILES that provide surviving stellar-mass fractions, FitSpec can distinguish formed-mass and present-day stellar-plus-remnant mass quantities where the metadata permit this interpretation.

5.16 Bayesian stellar-population inference

The deterministic variable-projection solution remains the first stage of stellar fitting, but FitSpec now also provides a stellar-specific adapter to the shared inference layer in `stellar/inference.py`. The implementation preserves an important mathematical distinction: repeatedly re-solving the non-negative SSP coefficients while sampling only dust and kinematics produces a *profile posterior*; it does not marginalize over stellar-population weights. FitSpec therefore exposes that approximation only under the explicit `profile` mode and labels its posterior metadata `profile_not_marginalized`.

The recommended `conditional` mode instead samples the nonlinear stellar parameters and the non-negative stellar-population amplitudes together. For a selected SSP basis $\mathcal{B}$, the sampled model is

$$M(\lambda) = \sum_{j \in \mathcal{B}} 10^{\mu_j} S'_j(\lambda; E(B-V), v_*, \sigma_*) + \sum_{k \in \mathcal{G}} a_k G_k(\lambda; v_{\text{gas}}, \sigma_{\text{gas}}), \quad (22)$$

where $\mu_j = \log_{10}(m_j/M_\odot)$ is sampled with a finite uniform prior, equivalent to a proper log-uniform prior on formed stellar mass over the configured interval. The nuisance-gas amplitudes are sampled only when requested and remain non-negative. Thus the resulting distribution is a full posterior over the parameters included in the selected stellar/gas basis, conditional on that basis choice.

Sampling every SSP in a large synthesis library can create an unnecessarily high-dimensional posterior. FitSpec therefore provides three basis-selection modes:

active sample only SSPs carrying non-zero weight in the deterministic solution;

candidate retain those active SSPs and add alternative age-metallicity templates whose existing single-SSP diagnostic satisfies a configured $\Delta\chi^2$ threshold;

full sample every SSP in the currently loaded library.

The **candidate** mode is the default because it permits the posterior to move into plausible age–metallicity alternatives that received zero deterministic NNLS weight while avoiding a blindly enormous sampling dimension. A configurable maximum number of SSP templates is enforced for every posterior basis; requesting **full** mode above this cap raises an error rather than silently launching an impractical calculation.

The complete loaded library remains scientifically relevant even in candidate mode: the candidate set is constructed from diagnostics evaluated over the deterministic full basis. Consequently the conditional posterior should be interpreted as

$$p(\boldsymbol{\theta}_*, \mathbf{m}_B, \mathbf{a}_G \mid \mathbf{F}, \mathcal{B}, \mathcal{G}), \quad (23)$$

not as a proof that excluded SSPs have exactly zero posterior probability. For analyses where posterior support across the complete library is essential, the user should reduce the supplied library to a scientifically defensible grid and use **full** mode.

The nonlinear parameters use the same finite physical bounds as deterministic fitting. Stellar masses use the configured `stellar_inference_log10_mass_bounds`. Nuisance-gas amplitudes may use an explicit finite upper prior; if none is supplied, FitSpec uses the same finite data-scaled ceiling employed by deterministic nuisance fitting. Because a data-scaled prior is not appropriate for absolute evidence interpretation, an explicit gas-amplitude prior is recommended whenever dynesty evidence will be compared between stellar models.

Both emcee and dynesty consume the same stellar `PosteriorProblem`. The posterior product stores its basis indices, posterior type, library family, regime, priors, and sampling metadata. The helper `stellar_population_samples` converts sampled log masses into total formed mass and per-SSP mass fractions for downstream population summaries. The standalone stellar GUI exposes the same Posterior/Load/Save/Plot workflow as the emission and absorption panels, while deterministic fitting remains the default unless `stellar_inference_method` is explicitly set to **emcee** or **dynesty**.

5.17 Saved stellar-fit and plotting products

The stellar result is saved independently of the GUI. The FITS product contains, at minimum,

- rest-frame wavelength;
- fitted data and uncertainty;
- exact fitting mask;
- total model and stellar model;
- nuisance gas model when enabled;
- fitted nonlinear parameters and local uncertainties;
- one row per SSP containing age, metallicity, coefficient, mass/light information, and dominant-component flags;
- stellar-library identity, normalization, and branch metadata;
- resolution provenance;
- fit diagnostics required to reconstruct the degeneracy plots.

The deterministic stellar `Plot` action reproduces the established standalone plotting workflow and saves all standard sidecars rather than displaying only one figure. For a result stem `<target>_res`, the products are

```
<target>_res.pdf
<target>_res_summary.pdf
<target>_res_diag.pdf
<target>_res_diag.txt
```

No code-version suffix such as `_v8` is part of the FitSpec naming convention. The main PDF contains the fitted spectrum/residual/population summary, the summary PDF contains the UV or optical observational feature panels and population-weight summary, and the diagnostic PDF retains the six-panel SSP-degeneracy layout: full correlation matrix, single-SSP $\Delta\chi^2$ landscape, fitted mass distribution, dominant-SSP correlations, transformed degenerate SSP spectra, and numerical summary. The diagnostic text file is saved with the fit/plot products. The plotting layer reads the saved fit products rather than recomputing the science fit.

5.18 What is new in the FitSpec stellar approach

The principal methodological novelty of the FitSpec stellar module is not the invention of SSP synthesis, attenuation curves, or least-squares optimization individually. Rather, it is their integration into one consistent UV–optical architecture:

1. multiple heterogeneous stellar synthesis libraries are converted into a common HDF5 population interface;
2. UV and optical spectra use the same separable nonlinear fitting architecture;
3. age–metallicity SSP amplitudes are solved linearly and non-negatively at every nonlinear dust/kinematic trial;
4. instrumental resolution is applied through a common observation-side resolution model;
5. exact FitSpec masks are used directly in the objective;
6. nuisance gas can be fitted without turning stellar fitting into an emission-line analysis;
7. degeneracy, effective-rank, and population diagnostics are treated as first-class outputs rather than optional plotting afterthoughts.

This design is intended to make stellar fitting portable across wavelength regimes and synthesis libraries without reintroducing separate historical fitting backends.

5.19 Emission-Line Fitting

5.19.1 Overview and relationship to the rest of FitSpec

The emission module (`emission/`) is developed as an independent FitSpec science module. It consumes the common core infrastructure – spectrum representation, masking, resolution, generic explicit-component parameters, and the shared deterministic-fitting engine (Sections 3, 9) – but contributes no changes to, and has no dependency on, the unified stellar-fitting architecture of Section 5. Concretely, the module supplies exactly two things to the generic optimizer: (i) an explicit `core.parameters.ModelParameters` instance describing the free/fixed kinematic and amplitude

parameters, and (ii) a `model_func(wave, model_parameters)` closure that evaluates the physical model for those parameters. Everything downstream of that – bounded nonlinear least-squares optimization, uncertainty propagation from the Jacobian, and the standard goodness-of-fit statistics – is provided unmodified by `core.fitting.fit_deterministic` and `core.statistics`, exactly as for the stellar and (future) absorption modules. This mirrors the "separate models from inference" design principle: the emission module describes the science, and never re-implements optimization.

5.19.2 The nebular line list and its atomic provenance

The default line list (`data/nebular_emission_line_list.csv`) is a 243-transition compilation spanning the far-UV through the near-IR ($\sim 770\text{--}10,940\text{ \AA}$), covering permitted resonance lines, semiforbidden and intercombination lines, and collisionally-excited forbidden lines from H, He, and the common metal ions (C, N, O, Ne, Mg, Si, S, Ar, Ca, Fe, and others), together with the full hydrogen Lyman, Balmer, and Paschen series members present in the table. Every column of the source table is preserved verbatim on the `emission.lines.EmissionLine` record – lower/upper energy in eV, the split lower/upper electron configuration and term symbols, the $J_i\text{--}J_k$ label, the transition multipole classification (E1, M1, E2, or a mixed M1+E2 type), the creation ionization potential, the literature observation reference, and the doublet-grouping metadata – so that no information from the source table is lost even though only a subset of it (rest wavelength, and the doublet-tying fields discussed below) is consumed by the fitting code itself. A user-supplied replacement list in the identical CSV schema is accepted via the `emission_line_list_path` configuration key, per the FitSpec requirement that the emission module use a default line list while allowing user-supplied line information.

Line names are constructed deterministically from the table as `f"{ion}-{int(wave)}"` (spaces in the ion label replaced with underscores), reproducing the historical naming convention (e.g. [O III] 5006, H α 6562) and guaranteed unique within one loaded list via an automatic disambiguating suffix in the rare case of an integer-wavelength collision.

Fixed-ratio doublet ties: when a ratio is an atomic constant. Thirty-eight of the 243 lines are flagged in the source table as members of a doublet (sharing a `doublet_id`), but only some doublet pairs have a flux ratio that is fixed by atomic physics rather than by the local physical conditions of the emitting gas. FitSpec's loader (`emission.lines.load_emission_line_list`) encodes this distinction directly from the table's `physical_basis` column:

- **"f-value"** – permitted resonance doublets arising from a common lower level (e.g. C IV 1548/1551, N V 1238/1242, O VI 1031/1037, Si IV 1393/1402, Al III 1854/1862, Ne VIII 770/780). In the optically thin limit the intensity ratio of two lines sharing a lower level is fixed by the ratio of their oscillator strengths (equivalently, by the ratio of their statistical weight-transition-probability products), independent of density or temperature ([Osterbrock & Ferland, 2006](#)).
- **"A-value"** – forbidden or semiforbidden multiplets arising from a common *upper* level (e.g. [O III] 4959/5007, [N II] 6548/6583, [O I] 6300/6364, [Ne III] 3868/3967, O III] 1660/1666). Because both transitions depopulate the same upper level, their intensity ratio is fixed purely by the ratio of their spontaneous (Einstein *A*) transition probabilities and is likewise density- and temperature-independent in the low-density limit relevant to essentially all nebular and extragalactic applications ([Storey & Zeippen, 2000](#)). For the [O III] auroral pair specifically, FitSpec's table encodes the modern *A*-value ratio of 2.98 (rather than the older, frequently-quoted value of 3.0).

For every `doublet_id` group whose members satisfy one of the two conditions above, the loader designates the lowest-numbered `doublet_member` as the free ("primary") line and *ties* every other member's fitted amplitude to it as

$$F_{\text{follower}} = \left(\frac{r_{\text{follower}}}{r_{\text{primary}}} \right) F_{\text{primary}}, \quad (24)$$

where r is the table's `intrinsic_ratio` value for each member and F_{primary} is the primary member's free integrated-flux amplitude parameter. A tied line therefore never receives its own free amplitude parameter in the fitted `ModelParameters` – it is algebraically eliminated before the model is ever evaluated – which both reduces the dimensionality of the nonlinear problem and enforces the atomic-physics constraint exactly rather than approximately.

Doublets that must *not* be tied: density-diagnostic pairs. Twelve lines belonging to six doublet pairs are flagged `physical_basis = "environment dependent"` and are deliberately left *untied*, each retaining its own free amplitude parameter, despite being marked as doublets in the source table. These are collisionally-excited transitions from two close-lying but physically distinct upper levels with different critical densities, so their intensity ratio is itself a sensitive diagnostic of the local electron density rather than a fixed atomic constant. The list includes the classic density-diagnostic pairs [O II] 3726/3729 and [S II] 6716/6731 and [S II] 4068/4076, and the intersystem pair Si III] 1882/1892 and semiforbidden O IV] 1397/1399 ([Osterbrock & Ferland, 2006](#), Sec. 5.9). Hard-tying these pairs would silently discard exactly the observable FitSpec users are most likely to want to measure; leaving them untied is therefore a deliberate scientific choice encoded structurally in the loader, not an oversight.

Atomic-data provenance. The table's individual per-line observation references (preserved on `EmissionLine.observation_reference`) trace to the original published measurements cited in the source compilation. Where FitSpec's own documentation needs a single, citable, actively-maintained reference for the underlying atomic transition data (wavelengths, energy levels, and transition probabilities) as a cross-check, the NIST Atomic Spectra Database is the standard modern reference ([Kramida et al., 2023](#)).

5.19.3 Explicit-component kinematic architecture

Per the "explicit state beats implicit inference" design principle (Section 13), the number of kinematic components (`emission_n_components`) is always an explicit integer set by configuration or by the GUI's component-count control, never inferred from whether some other configuration value happens to be scalar- or list-valued – a real bug class in the legacy emission-line GUI, where the shape of `curvefit_v_init` silently determined the fitted component count. Each kinematic component is realized as one `core.parameters.Component` holding:

- one shared `velocity_kms` parameter and one shared `sigma_kms` parameter, applied identically to every line assigned to that component (full kinematic tying *within* a component – e.g. a "narrow" component and a separate "broad" component both fit the full active line set, each with its own single velocity and dispersion); and
- one `amp_⟨line_name⟩` integrated-flux parameter per untied line in the active line set (Section 5.19.2 above describes how tied lines are excluded from this list).

Kinematic tying *across* components. `emission_kinematics_mode` controls how multiple components' kinematics relate to one another, reproducing and generalizing the legacy `kinematics` keyword (`free/tied/fixed`):

- **free** (default) – every component's velocity and dispersion are independently free parameters of the nonlinear fit.
- **tied** – every component after the first has its `velocity_kms/sigma_kms` `Parameter.fixed` flag set at construction time, removing them from the free-parameter vector entirely; the shared `model_func` closure then copies component 0's current velocity/dispersion *value* onto every follower component immediately before each model evaluation. The net effect is a single kinematic system driving every component's amplitudes, implemented by direct value synchronization rather than by literally sharing one `Parameter` object across components (which would inflate the apparent free-parameter count and corrupt the covariance estimate from the optimizer's Jacobian).
- **fixed** – every component's velocity and dispersion are held at the configured initial guess and never entered into the fit at all (useful, for example, when a kinematic solution has already been determined from a higher-S/N line and is being held fixed while only line-flux ratios are of interest).

5.19.4 Line-profile model

Each line's contribution to a component's flux is an area-normalized Gaussian,

$$f_{\lambda}(\lambda) = \frac{F}{\sigma_{\lambda}\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{\lambda - \lambda_c}{\sigma_{\lambda}}\right)^2\right], \quad (25)$$

where F is the fitted integrated-flux amplitude (rather than peak-normalized, as the generic primitive `core.models.gaussian` is – emission-line amplitudes are conventionally reported and compared as integrated flux, e.g. for line-ratio diagnostics, not as profile peak height). The line center follows the spectrum's systemic redshift z combined with the component's residual velocity v via the relativistic Doppler formula,

$$\lambda_c = \lambda_{\text{rest}} (1 + z) \sqrt{\frac{1 + v/c}{1 - v/c}}, \quad (26)$$

and the observed-frame width combines the component's intrinsic velocity dispersion with the spectrum's instrumental resolution in quadrature at the line center,

$$\sigma_{\lambda}(\lambda_c) = \sqrt{\left(\frac{\lambda_c \sigma_v}{c}\right)^2 + \sigma_{\text{inst}}^2(\lambda_c)}, \quad (27)$$

where $\sigma_{\text{inst}}(\lambda_c)$ is drawn from the spectrum's `core.resolution.ResolutionModel`. This directly enforces the "resolution must never be silently ignored" design principle for the emission module: every fitted line width is always resolution-aware, and if no resolution model was supplied for the spectrum, the upstream `DefaultResolutionWarning` mechanism in `core.resolution` is responsible for surfacing that fact rather than the emission module silently proceeding with an intrinsic-only width.

5.19.5 Line-list selection

Which lines from the catalog actually enter a given fit is resolved by `emission.emission_fit.select_emission_line` with the following precedence, matching and formalizing the legacy behavior:

1. If `emission_lines` (an explicit comma-separated subset of line names) is set, it is used exactly as given.
2. Otherwise, if either `emission_strong_lines` or `emission_weak_lines` is set, their union is used.
3. Otherwise, every catalog line whose observed-frame wavelength $\lambda_{\text{rest}}(1+z)$ falls within the spectrum's wavelength coverage is selected automatically – reproducing the legacy default of generating the fitted line set from the spectral range rather than requiring it to be listed explicitly.

In every case, if a selected line is tied to a line outside the selected subset, the tied-to line is pulled in automatically by `emission.lines.select_lines`, since a tie cannot reference an amplitude parameter that does not exist in the fit.

5.19.6 User-specified fixed amplitude ratios

Beyond the catalog's own atomic-constant ties (Section 5.19.2), a user may fix the amplitude ratio between *any* two lines at will – including a pair the catalog deliberately leaves untied, such as the density-diagnostic [S II] 6716/6731 or [O II] 3726/3729 doublets – via `emission_fixed_ratio_species/emission_fixed_ratio_value` in configuration. This is intended primarily for cross-checking a published line-ratio value: fixing a diagnostic doublet's ratio to a literature value and comparing the resulting fit (e.g. its χ^2 , or the other lines' recovered fluxes) against a free fit of the same data is a natural consistency check that the rest of the model cannot perform on its own.

`emission_fixed_ratio_species` takes a list of line-name pairs in a nested-bracket notation, and `emission_fixed_ratio_value` one positive ratio per pair, in the same order:

```
emission_fixed_ratio_species = [[S_II]6716,[S_II]6730], [[O_II]3726,[O_II]3728]
emission_fixed_ratio_value = 1.1, 0.9
```

Each ratio is defined as `flux(first-listed line) / flux(second-listed line)`; reversing a pair's order fixes the ratio the other way around. Line names must match the catalog's generated names exactly (Section 5.19.2); note in particular that FitSpec's names retain an underscore for the ion's internal space (e.g. "[S_II]6716", not "[SII]6716").

Because FitSpec's own generated line names themselves contain literal square brackets (the ion notation), a naive comma-delimited parse of this configuration value cannot distinguish a pair's own delimiting brackets from brackets that are part of a name, and `core.config`'s generic list-inference additionally splits on every comma before the module-specific parser ever sees the string. `emission.emission_fit._parse_species_pairs` therefore reconstructs the original text (rejoining any naive split, which is lossless since it exactly reverses a plain comma split regardless of the text's meaning) and re-parses it with a bracket-depth-aware tokenizer, so nested brackets are resolved unambiguously rather than by position or heuristic.

At the model level, this is implemented by `emission.lines.apply_fixed_ratio_overrides`, applied to the full catalog by `select_emission_line_list` before any wavelength-range or explicit-list subsetting resolves its tie closure (Section 5.19.2), so a user-specified tie is correctly pulled in as a dependency exactly like a catalog tie would be. A user override may also supersede an existing

catalog tie for the same pair (e.g. substituting a different assumed [O III] 4959/5007 ratio for a specific test), and the combined set of ties – catalog plus user overrides – is checked for cycles before fitting, since a self-referential or mutually-inconsistent set of ties (e.g. line A tied to B and B tied to A) has no well-defined amplitude assignment.

5.19.7 Flux pre-scaling

Two legacy pre-fit transformations are retained as optional, fully-invertible rescalings applied immediately before the optimizer runs and undone on the returned result: `emission_flux_reduction` (a flat subtractive offset, legacy `flux_red`) and `emission_flux_normalizing_factor` (a multiplicative divisor applied to both flux and its uncertainty, legacy `flux_normalizing_factor`), i.e. the optimizer sees $f' = (f - f_{\text{red}})/N$ and every reported flux, model array, and amplitude parameter (and its propagated uncertainty) is scaled back by the same factor before being returned to the caller. This exists purely as a numerical-conditioning aid for optimizers that are sensitive to the absolute magnitude of the residual vector; it changes no physical result.

5.19.8 Automatic component-count selection

When `emission_n_components_max` is set above `emission_n_components`, FitSpec performs the deterministic fit independently for every component count n in that range and selects the count minimizing

$$\text{BIC}(n) + \kappa n, \quad (28)$$

where $\text{BIC}(n) = k(n) \ln N_{\text{data}} - 2 \ln \hat{L}(n)$ is the standard Bayesian Information Criterion (Schwarz, 1978) already computed by `core.statistics.compute_fit_statistics` for every fit (with $k(n)$ the number of free parameters at that component count and $\hat{L}$ the fit’s maximized likelihood), and κ is `emission_bic_penalty_factor` (legacy `penalty_factor`), an additional configurable per-component penalty beyond the BIC’s own complexity term – retained from the legacy fitting script’s automatic multi-component search, where it was used to further discourage marginal additional components beyond what the BIC’s asymptotic penalty alone would reject.

5.19.9 Fitting engine and results

Fitting itself reuses the shared `core.fitting.fit_deterministic` engine unmodified: the resulting bounded nonlinear least-squares problem (all kinematic and amplitude parameters, whether tied indirectly via Eq. 24 or via the cross-component synchronization described above, fit jointly) is solved with SciPy’s `curve_fit` machinery (Virtanen et al., 2020), and parameter uncertainties are derived from the fit’s covariance matrix in the same way as for every other FitSpec science module. Results are summarized (`emission.emission_results.summarize_emission_fit`) into per-line integrated fluxes – with uncertainty on a tied line’s flux propagated through the fixed ratio in Eq. 24 as $\sigma_{\text{follower}} = (r_{\text{follower}}/r_{\text{primary}}) \sigma_{\text{primary}}$ – and per-component kinematics, and round-trip to FITS via `save_emission_result/load_emission_result`, mirroring the stellar module’s save/load convention (Section 5). Non-ASCII ion labels arising from Greek-letter line names (e.g. $\text{H}\alpha$, $\text{Ly}\alpha$) are round-tripped losslessly through the FITS ASCII-table format via backslash-escaping, since the FITS ‘A’ column type does not itself support non-ASCII characters.

5.20 Posterior inference integration

The emission module is now connected directly to the shared posterior framework of Section 7. Posterior sampling does not replace the deterministic emission fit. FitSpec first performs the

deterministic fit, including any configured BIC component-count search, and treats the selected component structure, line list, fixed-ratio constraints, kinematic tying mode, mask, redshift, and instrumental resolution as the physical model to be sampled. This separation prevents posterior parameter estimation from being conflated implicitly with component-number selection.

The science-module adapter is implemented in `emission/inference.py`. The public entry points are

- `build_emission_prior_set`: constructs proper priors for the free parameters of the selected deterministic model;
- `build_emission_posterior_problem`: reconstructs the shared `PosteriorProblem` while preserving the emission line list, kinematic tying mode, deterministic mask, redshift, instrumental resolution, and native-unit flux convention;
- `run_emission_inference`: dispatches the problem to `emcee` or `dynesty` using the emission inference configuration.

New emission FITS products also preserve every parameter value, bound, and fixed/free flag, plus the active component and the emission-specific scaling/kinematic metadata required to reconstruct the posterior state. Older result files remain readable, but bounds that were never stored in those older files necessarily fall back to infinite values and therefore require an explicit proper prior before posterior sampling.

The deterministic emission fitter may rescale the flux numerically before optimization through `emission_flux_normalizing_factor` and `emission_flux_reduction`. The stored `FitResult` is restored to the spectrum's native flux units. Posterior construction therefore operates in those same native units, restores the additive flux offset in every model evaluation, and uses native-unit amplitude values and bounds. When a finite deterministic amplitude bound is used together with flux normalization, the bound is transformed back to native units together with the fitted amplitude; this keeps the deterministic parameter state and subsequent Bayesian prior numerically consistent.

For velocity and dispersion, finite deterministic parameter bounds become uniform priors by default. Emission amplitudes require special care because the deterministic default permits an infinite upper bound. An infinite optimizer bound is not a proper Bayesian prior. Therefore posterior sampling requires either a finite `emission_maximum_amplitude` or a finite positive `emission_inference_amplitude_prior_max`. FitSpec raises an explicit error rather than silently inventing a data-dependent or improper prior.

Posterior execution is controlled by `emission_inference_method`, whose allowed values are `deterministic`, `emcee`, and `dynesty`. The default is `deterministic`, which leaves posterior sampling disabled and introduces no runtime dependency on either optional sampler. The `emcee` configuration exposes walker count, total steps, burn-in, thinning, and initial-cloud scale. The `dynesty` configuration exposes static/dynamic selection, live-point count, evidence stopping tolerance, sampling strategy, and bounding strategy. A common random seed and terminal progress switch are provided for reproducibility and interactive use.

The emission GUI keeps deterministic and posterior actions visibly separate. `Fit` performs the deterministic fit; `Posterior` samples the already-selected model using the configured backend; `Load P` and `Save P` restore/write the portable compressed `PosteriorResult` product; and `Plot P` opens the shared corner-style posterior diagnostic. The terminal prints the 16th, 50th, and 84th percentile summary for every sampled parameter. This design makes it explicit to the user whether displayed uncertainties are local Jacobian-covariance errors from the deterministic optimizer or marginalized posterior credible intervals from a sampler.

A minimal terminal workflow is

```

from emission import fit_emission_spectrum, run_emission_inference

fit = fit_emission_spectrum(spectrum, config)
posterior = run_emission_inference(fit, spectrum, config)

if posterior is not None:
    posterior.save_npz("emission_posterior.npz")

```

At this stage posterior inference conditions on the deterministic model structure. Bayesian evidence comparison between different emission component counts can be performed explicitly with dynesty in a future model-comparison driver, but FitSpec does not automatically reinterpret a parameter posterior run as trans-dimensional inference.

5.21 Absorption-Line Fitting

5.21.1 Overview

The absorption module (`absorption/`) is developed as an independent FitSpec science module, sharing the common core infrastructure and the generic deterministic-fitting engine (Section 3) but contributing no changes to the stellar or emission modules. As with those modules, the absorption module's only responsibility is to supply an explicit `core.parameters.ModelParameters` instance and a `model_func(wave, model_parameters)` closure to `core.fitting.fit_deterministic`; the optimizer, uncertainty propagation, and fit statistics are unmodified shared infrastructure.

5.21.2 Atomic transition data

The default atomic line list (`data/atomic_absorption_lines.csv`) is derived in full from the atomic-data compilation distributed with the VPFIT Voigt-profile fitting program (Carswell & Webb, 2014), itself built substantially on Morton's standard reference compilation of atomic data for resonance absorption lines (Morton, 2003) together with contributions and updates from numerous individually-cited sources tracked per-transition in the source file. Every genuine transition line in the source `atom.dat` is retained: 1319 transitions across 127 distinct species/groups, spanning ordinary atomic resonance lines (H through the iron-group and beyond), isotopic variants (D I, ^{13}C , ^6Li), excited fine-structure levels (e.g. Si II*, C II*, Fe II*), and molecular electronic-band systems (H_2 , CO, HD rotational levels) – none of which required any change to the underlying physics, since Eq. 29 is a generic Voigt optical-depth expression parameterized only by wavelength, oscillator strength, and damping constant, indifferent to whether the transition originates in an atom, ion, or molecule. VPFIT-internal bookkeeping "species" that are not real transitions (region markers, continuum/zero-level adjustment flags, disabled/commented-out entries prefixed !) are excluded during parsing, and the small number of source-file typos encountered (a doubled exponent marker in one damping constant) were repaired rather than silently dropped. A transition's free-text provenance note from the source file (literature reference, calibration caveat, isotope-shift remark) is preserved verbatim on `AtomicTransition.reference`, so that – exactly as for the emission line list (Section 5.19.2) – no information from the source compilation is lost even though only a subset of it (rest wavelength, oscillator strength, damping constant, and grouping) is consumed by the fitting code itself. A small number of transitions carry a damping constant of exactly zero, where no radiative damping rate has been measured or tabulated for that line; FitSpec treats this as physically valid (the Voigt-Hjerting function reduces smoothly to a pure Gaussian, Doppler-core-only profile in that limit) rather than raising an error. As with the emission line list, a user-supplied replacement

in the identical CSV schema is accepted via `absorption_line_list_path`, per the requirement that the absorption module use a default atomic-data file while allowing user-supplied atomic data. Wavelengths in this compilation are vacuum throughout (no air/vacuum split at 2000 Å, unlike the nebular emission-line list); see Section 3 for the medium-handling implications.

Transitions are grouped (`AtomicTransition.group`) into physical systems – every H I Lyman-series line shares group "HI", both C IV lines share group "CIV", and so on – because transitions in the same group arise from the same absorbing gas and must always be fit together with one shared column density, Doppler parameter, and velocity per kinematic component. Groups are derived directly from the source compilation's own species labels (whitespace removed), which means an excited fine-structure level is automatically kept as its own separate group from the corresponding ground state – e.g. "SiII*" never gets merged into "SiII" – since the two represent physically distinct populations that may carry different column densities even when they share the same kinematics. This is exactly how VPFIT itself groups transitions by species label for a fitting region. `absorption_group` in configuration selects which group is fit (e.g. "CIV"); `absorption.atomic.select_group` resolves it to the corresponding transition list, and `absorption.atomic.list_groups` enumerates every group available in a loaded line list.

5.21.3 Explicit-component kinematic architecture

As in the emission module, the number of kinematic (velocity) components is always an explicit `absorption_n_components` rather than inferred implicitly. Each component is one `core.parameters.Component` representing a single absorbing "cloud" along the line of sight, holding three parameters applied identically to every transition in the active group: `logN` ($\log_{10}$ column density, cm^{-2}), `b_kms` (Doppler parameter), and `velocity_kms` (residual velocity relative to the spectrum's systemic redshift). Column density is fit in log space – standard practice, since N spans many orders of magnitude with highly asymmetric uncertainty in linear space – and converted to the linear N that actually enters the optical-depth calculation only inside the model evaluation itself.

5.21.4 Optical-depth model

For one transition and one velocity component, the absorption optical depth as a function of wavelength is

$$\tau(\lambda) = \frac{\sqrt{\pi} e^2}{m_e c} f \lambda_0 \frac{N}{b} H(a, x), \quad (29)$$

where f is the oscillator strength, λ_0 the rest wavelength, N the column density, b the Doppler parameter, and $H(a, x)$ the Voigt-Hjerting function with damping parameter $a = \lambda_0 \Gamma / (4\pi b)$ (Gamma the transition's radiative damping constant) and dimensionless offset $x = (\lambda_{\text{rest}} - \lambda_0) / \Delta\lambda_D$ from line center in Doppler-width units, $\Delta\lambda_D = (b/c)\lambda_0$. FitSpec evaluates $H(a, x)$ with the Tepper-Garcia (2006) rational approximation (Tepper-García, 2006), accurate to better than 1% across the full range of column densities and Doppler parameters relevant to resonance absorption-line fitting, and avoids an explicit Faddeeva-function evaluation (used instead by the generic, unrelated Voigt primitive in `core.models`, which is peak-normalized and dimensionless rather than physically parameterized by N , b , f , and Γ). A component's contribution to a multi-transition group's total optical depth is the sum of Eq. 29 over every transition in the group; a fit's full model is the sum of every component's contribution, converted to transmission as $T_{\text{full}} = \exp(-\tau_{\text{total}})$.

5.21.5 Partial covering

By default (`absorption_partial_coverage = False`), FitSpec assumes the absorbing gas fully covers the background continuum source – the standard simplifying assumption, and the only physically meaningful option when the background source is a single unresolved point (e.g. direct stellar continuum absorption). This default costs nothing extra: no covering-fraction parameter is added to the fit at all, so a full-coverage fit with n components has exactly $3n$ free parameters.

When `absorption_partial_coverage` is set to true, FitSpec instead fits the standard partial-covering model

$$T_{\text{obs}}(\lambda) = (1 - C_f) + C_f T_{\text{full}}(\lambda), \quad (30)$$

where $C_f \in [0, 1]$ is the covering fraction: the fraction of the background source’s light that passes through the absorbing gas along a sightline that intercepts it, with the remaining $(1 - C_f)$ fraction reaching the observer unabsorbed. $C_f = 1$ recovers full coverage exactly. This formalism is standard in studies of AGN/quasar intrinsic and associated absorption, where the background continuum source (accretion disk, or disk plus broad-line region) can have finite angular size relative to the absorbing outflow, so that the absorber covers only part of it (Barlow & Sargent, 1997; Ganguly et al., 2003; Arav et al., 2005); it is the same model implemented and validated against synthetic doublet data in the FitSpec partial-coverage reference scripts.

C_f is a single parameter shared across every kinematic component – not one C_f per component – because covering geometry is a property of the alignment between the background source and the absorbing outflow/cloud complex as a whole, not of any individual velocity component within it. Structurally, this is implemented by adding a free `covering_fraction` parameter only to component 0; every other component gets the same-named parameter held `fixed=True` (excluded from the free-parameter vector, so it never inflates the apparent parameter count or corrupts the optimizer’s covariance estimate) and synchronized to component 0’s current value at every model evaluation – the identical cross-component-tying mechanism used for kinematics in the emission module (Section 5.19).

Evaluation order. The mixing in Eq. 30 is a property of the source geometry, intrinsic to the astrophysical system, and is applied to the *unconvolved* transmission; the spectrum’s instrumental resolution is convolved with the resulting observed-frame transmission only afterward, since instrumental broadening is a purely observational effect layered on top of whatever the telescope actually receives. FitSpec’s `make_absorption_model_func` therefore evaluates, in order: (1) sum every component’s optical depth over every transition in the group and exponentiate to T_{full} ; (2) apply Eq. 30 if partial coverage is enabled; (3) convolve with instrumental resolution via the same `core.resolution.convolve_variable_gaussian` machinery used by the stellar module, enforcing the "resolution must never be silently ignored" design principle for absorption fits exactly as for the other science modules.

5.21.6 Fitting engine and results

Fitting reuses `core.fitting.fit_deterministic` unmodified, exactly as for the stellar and emission modules; parameter uncertainties come from the same Jacobian-based covariance estimate. Results are summarized (`absorption.absorption_results.summarize_absorption_fit`) into one `AbsorptionComponentMeasurement` per kinematic component ($\log N$, b , and velocity with their uncertainties) plus, when partial coverage was used, the single shared covering fraction and its uncertainty, and round-trip to FITS via `save_absorption_result/load_absorption_result`, mirroring the stellar and emission modules’ save/load convention.

5.21.7 Posterior inference for single-group absorption fits

The deterministic solution remains the required first stage of an absorption fit. It defines the number of kinematic components, the active transition group, the fitted mask, the instrumental-resolution context, the partial-covering choice, and the best-fit starting point. The finalized deterministic result can then be passed to `absorption.inference.run_absorption_inference`, which converts the same physical model into the common `inference.PosteriorProblem` described in Section 7 and dispatches it to `emcee` or `dynesty`. Posterior sampling therefore estimates uncertainty within an explicitly accepted absorption model; it does not silently add or remove velocity components while sampling.

For an ordinary single-group fit the complete workflow is

```
from absorption import fit_absorption_spectrum, run_absorption_inference

fit = fit_absorption_spectrum(spectrum, config, transitions=transitions)
posterior = run_absorption_inference(
    fit, spectrum, config, transitions=transitions,
)
```

The configuration key `absorption_inference_method` selects `deterministic`, `emcee`, or `dynesty`. Separate configuration groups control walker count, chain length, burn-in/thinning, live points, nested-sampling tolerance, random seed, progress reporting, posterior output path, and the maximum number of dimensions shown in the default corner diagnostic. The GUI exposes the posterior stage independently through `Posterior`, `Load P`, `Save P`, and `Plot P` controls, so running an optimizer never implicitly launches a potentially expensive sampler.

Absorption parameters already have finite physically defined bounds in the standard model builders: `logN`, `b_kms`, `velocity_kms`, and, when enabled, `covering_fraction`. The default posterior therefore assigns an independent uniform prior over each free parameter’s existing deterministic bounds. A parameter marked `fixed=True` is excluded from the posterior vector exactly as it is excluded from deterministic optimization. FitSpec refuses to construct a posterior from a free absorption parameter with an infinite bound rather than silently treating an improper optimizer range as a Bayesian prior.

Partial covering is sampled without any special approximation. Component 0’s free `covering_fraction` remains a posterior dimension, while the fixed copies carried by the remaining components are synchronized from the leader at every model evaluation. The same subpixel optical-depth oversampling used for the deterministic fit is also used by the posterior model, so posterior likelihoods are evaluated with the same narrow-line numerical resolution as the accepted best fit.

The absorption FITS result format now persists the full `ModelParameters` state in addition to the derived component table: every parameter’s value, lower and upper bound, fixed/free flag, uncertainty, active-component index, fit mode, subpixel setting, and component-to-transition mapping are retained. Consequently a newly written deterministic absorption result remains inference-ready after being closed and reloaded. Older FitSpec result files remain readable for backward compatibility, but bounds that were never stored in those files cannot be reconstructed; such a legacy result must be refit (or otherwise supplied with proper priors) before posterior sampling.

5.22 Advanced Absorption Features

Following a review of the VPFIT 12.3 and RDGEN 12.3 manuals (Carswell & Webb, 2014), the absorption module was extended with the features below, each a close analogue of a documented

VPFIT/RDGEN capability adapted to FitSpec’s explicit-component architecture. Joint fitting across *separate* spectra/files was explicitly out of scope for this pass and is not implemented; every feature here still operates on one shared spectrum.

5.22.1 A generic parameter-tying primitive

`core.parameters.ParameterTie` generalizes the ad hoc cross-component tying already used for the emission module’s kinematics (Section 5.19) and the absorption module’s covering fraction (Section 5.21.5) into a single reusable primitive, available to every science module. A tie specifies a leader (`component_index`, `parameter_name`), a follower of the same form, and an optional transform function; `core.parameters.apply_ties` synchronizes every follower’s value from its leader, in list order, once per model evaluation. The follower’s underlying `Parameter.fixed` flag must already be set, so the optimizer never treats it as an independent free dimension – ties are resolved entirely inside the `model_func` closure and are invisible to `core.fitting.fit_deterministic`, exactly like every other science-specific modeling choice. Ties may be chained, since a parameter that is itself a follower of one tie may act as the leader of another, provided `apply_ties` processes them in dependency order.

5.22.2 Joint, multi-system (cross-ion) fitting

`absorption.absorption_model.AbsorptionSystem` packages one transition group’s kinematic-component block (its transitions, component count, and initial guesses/bounds); `build_joint_absorption_parameters` concatenates several systems’ components into one joint `ModelParameters`, returning a parallel `component_transitions` list recording which system’s transitions apply to each joint-numbered component. `make_joint_absorption_model_func` evaluates the joint model: every component contributes its optical depth using only its own system’s transitions, but any number of components across different systems can be linked via `ParameterTie` (e.g. forcing two ions to share one redshift), matching VPFIT’s own practice of fitting several species in a single `fort.13` start file with tied columns. `absorption.absorption_fit.fit_joint_absorption_spectrum` is the corresponding high-level entry point, taking a list of `AbsorptionSystem` objects plus optional tie specifications and returning an `AbsorptionFitResult` whose per-component measurements are each tagged with their originating system’s label. Because these cross-system linkages are inherently code-level constructs – VPFIT itself requires a hand-crafted start file for the same reason – this entry point is programmatic rather than config-string-driven, unlike the single-group `fit_absorption_spectrum`.

5.22.3 Thermal and turbulent Doppler-parameter linking

Two ions of different mass sharing one physical cloud need not share one Doppler parameter: under a combined turbulent-plus-thermal broadening model, $b^2 = b_{\text{turb}}^2 + b_T^2$ with $b_T = 12.85\sqrt{T/10^4\text{ K}/m}\text{ km s}^{-1}$ (m in amu) – VPFIT’s own constant, reproduced exactly in `absorption.absorption_model.ThermalTurbulentLink` ties a follower component’s `b_kms` to an assumed shared turbulent velocity and temperature, computed via `apply_thermal_turbulent_links`; `mass_scaled_b_transform` provides the simpler, purely-thermal limit ($b_{\text{turb}} = 0$) as an ordinary `ParameterTie` transform. This requires the atomic mass, now parsed from the source `atom.dat` compilation onto `AtomicTransition.atomic_mass` (with VPFIT’s own carry-forward convention: a species’ mass, once given, applies to every subsequent line of that species that omits it).

5.22.4 Fixed-ratio and common-pattern column-density ties

`fixed_log_ratio_transform` gives a `ParameterTie` enforcing $\log N_{\text{follower}} = \log N_{\text{leader}} + \Delta$ for a caller-chosen constant Δ – a fixed isotope ratio, or an assumed fixed molecular-level population pattern (VPFIT’s `vp_abund.dat` mechanism). `AbundancePatternGroup` implements the more general "common pattern relative ion abundances" case, where the ratio itself is a free fit parameter: every paired follower component’s $\log N$ is forced to track its leader’s $\log N$ plus one shared, fitted `logN_ratio`, so the velocity-component-by-component pattern between two ions is locked while their overall relative normalization floats. On synthetic two-component C IV/Si IV data with a true -0.5 dex offset, this recovers both components’ column densities to within 0.1 dex and the fitted ratio to -0.49 ± 0.1 dex against the true -0.50 .

5.22.5 Subpixel profile oversampling

`make_absorption_model_func/make_joint_absorption_model_func` accept a `subpixel` factor: the optical depth is evaluated on a grid oversampled by that factor (linear interpolation between the data’s own wavelength points), convolved with instrumental resolution on the fine grid, and resampled back onto the data grid, avoiding aliasing for lines narrow relative to the pixel scale. `subpixel=1` (the default) reproduces the previous direct-evaluation behavior exactly.

5.22.6 Region velocity shifts and in-fit continuum adjustment

`RegionShift` reproduces VPFIT’s region-wavelength-shift mechanism (its " $\gg$ " pseudo-species): a fittable velocity offset applied only within a specified wavelength window, for a possible calibration mismatch between two parts of a spectrum. `ContinuumAdjustment` reproduces the " $<>$ " mechanism: a small, fittable multiplicative level-plus-slope correction to the continuum within a window, intended for minor adjustments in crowded regions (e.g. the Ly α forest) rather than as a substitute for proper continuum fitting (Section 6) – a free continuum level can otherwise trade off against a broad, shallow absorption feature if left unconstrained. Both are applied via dedicated `apply_region_shifts/apply_continuum_adjustments` functions rather than the generic `ParameterTie` mechanism, since they act on the wavelength axis and the combined transmission respectively rather than syncing one parameter’s value from another.

Zero-level adjustment. VPFIT additionally implements partial covering itself via a repurposed zero-level-adjustment pseudo-species ("`__`"), stretching the fitted profile toward a nonzero floor – mathematically equivalent to the dedicated `covering_fraction` parameter already described in Section 5.21.5. FitSpec does not duplicate VPFIT’s literal parameterization, since the two are the same physics reached by different routes.

5.22.7 The unidentified-line marker

The atomic-data catalog (Section 5.21) retains VPFIT’s unidentified-line marker (species "`??`", by convention carrying the same oscillator strength/damping constant as Ly α) as an ordinary, fittable group, useful for modeling an unresolved blend as a free line without committing to a species identification.

5.22.8 Automatic rejection of insignificant components

`absorption.rejection.fit_joint_absorption_spectrum_with_rejection` is a simplified analogue of VPFIT’s `dropmode`: fit, identify components the data do not actually require, and refit

with those components no longer free. Unlike VPFIT's own **dropmode**, an insignificant component is never deleted from the model – it is instead *frozen*: its $\log N$, b , and velocity are all held fixed at their current (already negligible or degenerate) values, excluded from the free-parameter vector, and it continues to contribute its now-fixed, negligible optical depth to the model for every subsequent pass. This is a deliberate departure from VPFIT, made for two reasons: (i) the returned fit then always has exactly the number of components the caller originally requested, with no bookkeeping required to track a shrinking count across passes; and (ii) the summed column density across a system's components is essentially unaffected by rejection, since a frozen component's contribution was already negligible (or, in a degenerate blend, its removal would have redistributed rather than genuinely reduced the total) at the point it was frozen. A frozen component's $\log N$ should accordingly be read as an upper limit on its true column density rather than a detection, and is marked as such (`AbsorptionComponentMeasurement.is_upper_limit`).

A component is frozen if its fitted $\log N$ is pinned within a configurable margin of the system's column-density floor, or if its uncertainty is non-finite or unreasonably large – the numerical signature of a narrow, negligible-amplitude "phantom" component, or of a genuinely degenerate multi-component blend (several near-identical components sharing one real absorption feature, individually unconstrained even though their sum is well determined), that did not necessarily land exactly on the floor but was never really constrained by the data. Every *unfrozen* component's fitted value from the previous pass is carried forward as the next pass's starting point, so freezing only removes degrees of freedom without disturbing the fit of the components that remain free. A system's components are never all frozen at once: if every component in a system would be frozen, the single most significant one (highest $\log N$) is kept free. This still differs from VPFIT's own **dropmode** in one respect: it operates independently within each system rather than across an arbitrary set of cross-system ties, since renumbering ties after a component's role changes is not generally well-defined without knowing the caller's intent – combining rejection with cross-system tying is left to direct, un-rejected use of `fit_joint_absorption_spectrum`.

5.22.9 Column-density upper limits

`absorption.upper_limits.estimate_column_density_upper_limit` implements the method described in RDGEN's documentation for placing a rigorous upper limit on a non-detected ion given a redshift and Doppler parameter already determined from a reference ion, attributed there to [Schaye et al. \(2007\)](#). The reference ion's b (with its uncertainty) is decomposed into fully-turbulent and fully-thermal extremes to bound a plausible range of Doppler parameters for the test ion (via the same mass-scaling as Section 5.22.1); for each trial b in that range, the column density is increased from a starting value until the predicted absorption profile becomes statistically inconsistent with the data – specifically, until the χ^2 computed over pixels where the trial profile predicts deeper absorption than the data permit (within a window of the line center, requiring a minimum pixel count) falls below a chosen right-tail probability (0.16 by default, corresponding to roughly a one-sided 1σ limit). The adopted limit is the maximum $\log N$ found over the full b grid – the most conservative value, and one that remains valid in the presence of blends, since only pixels where the trial profile is too deep relative to the data ever contribute to the statistic. On synthetic pure-noise data this returns a modest, finite limit; when a real absorption line is present in the synthetic spectrum, the search correctly reports a much higher value, reflecting that weak or no absorption is no longer statistically consistent with the data. This is FitSpec's own implementation of the documented method, built on FitSpec's Voigt-profile machinery, not a port of VPFIT/RDGEN's Fortran implementation.

5.22.10 Synthetic spectrum generation

`absorption.synthetic.generate_synthetic_absorption_spectrum` reduces RDGEN's `gp/noise` workflow (generate Voigt profiles into the continuum array, then add Gaussian noise) to a single function call against FitSpec's own model machinery: given a transition group, a list of component parameter dictionaries, and either a target signal-to-noise ratio or an explicit noise level, it returns a `core.spectrum.Spectrum` with reproducible noise (via an explicit random seed) for testing recoverability or exploring detectability before committing telescope time.

5.22.11 Posterior inference for joint, tied, and rejected absorption models

The posterior layer preserves the advanced absorption model rather than replacing it with an unconstrained collection of Voigt components. For a joint cross-ion fit, `build_joint_absorption_posterior_problem` and `run_absorption_inference(..., systems=...)` rebuild the same component-to-transition assignment and reapply the same generic `ParameterTie`, `ThermalTurbulentLink`, and `AbundancePatternGroup` transformations on every likelihood evaluation. A tie follower is a derived quantity, not an independent degree of freedom: FitSpec now marks generic and thermal-link followers `fixed=True` before deterministic fitting, and the posterior adapter defensively enforces the same state when reconstructing older results. Abundance-pattern follower column densities are likewise excluded from the sampled vector while the shared fitted log-ratio holder remains free.

Because a generic `ParameterTie.transform` may be an arbitrary Python callable (including a user-defined closure), FitSpec deliberately does not attempt to serialize executable tie logic into a FITS file. Reproducible joint posterior sampling therefore requires the caller to pass the same `systems`, `ties`, `thermal_turbulent_links`, and `abundance_pattern_groups` objects used to define the deterministic model:

```
posterior = run_absorption_inference(
    fit, spectrum, config,
    systems=systems,
    ties=ties,
    thermal_turbulent_links=thermal_links,
    abundance_pattern_groups=abundance_groups,
)
```

The saved result still retains the fitted values, physical bounds, fixed/free state, system labels, transition mapping, and rejection flags; only the executable definition of an arbitrary code-level transform must be supplied again by the analysis script.

Automatic rejection and posterior sampling have intentionally different roles. The rejection stage identifies components the data do not require and freezes their `logN`, `b_kms`, and `velocity_kms` at their already negligible final values. Those components are tagged as upper limits/non-detections and remain fixed in the posterior; posterior sampling is not allowed to resurrect them as free detections after the accepted model has declared them insignificant. At least one component per system remains free by construction in the rejection algorithm.

The separate `estimate_column_density_upper_limit` procedure also remains conceptually distinct. It computes a conservative non-detection limit over an allowed Doppler-parameter range; it is not currently implemented as a censored-data Bayesian likelihood term. FitSpec therefore does not combine that reported upper limit with a sampled `logN` posterior as if the two were the same statistical object. A future censored-likelihood implementation, if scientifically required, should be introduced explicitly rather than inferred from the presence of an upper-limit flag.

6 Continuum Estimation

6.1 Motivation

Continuum estimation is a fundamental but often under-documented step in spectroscopy. A poorly estimated continuum can bias equivalent widths, emission-line fluxes, absorption-line column densities, profile shapes, and line-significance measurements. FitSpec therefore treats continuum estimation as a dedicated reusable science module rather than embedding continuum assumptions separately inside the emission and absorption fitters.

The continuum module is intentionally independent of the stellar-population fitter. In stellar fitting, the broad-band continuum is physically constrained by the SSP mixture and attenuation. In line fitting, by contrast, a local or phenomenological continuum estimate may be required before or during profile fitting. FitSpec therefore provides several interchangeable continuum estimators behind one interface.

The dispatcher

```
estimate_continuum(wave, flux, method=...)
```

returns a continuum defined on the supplied wavelength grid. The same continuum representation can be modified manually in the GUI or produced automatically.

6.2 The FitSpec custom continuum algorithm

The default `custom` method is a multi-stage robust continuum estimator developed within the project. It combines local smoothing, upper/lower-envelope information, automated broad-feature detection, derivative-based point weighting, percentile selection, Savitzky–Golay smoothing, and interpolation.

The historical source code contained a comment attributing inspiration to “Martin et al. 2021, 2021MNRAS.500.4937M.” That bibliographic identifier corresponds to an unrelated study of dwarf-galaxy mergers and is therefore *not* retained as methodological provenance here. The custom continuum method should be cited as a FitSpec method, while its established numerical building blocks are cited separately.

6.2.1 Step 1: finite-pixel selection

Non-finite flux values are removed from the working continuum sample. Let the finite spectrum be

$$\{\lambda_i, F_i\}_{i=1}^N. \quad (31)$$

This operation does not redefine the global FitSpec science mask; it only ensures that the continuum algorithm does not propagate NaN or infinite values through interpolation and filtering.

6.2.2 Step 2: preliminary boxcar smoothing

The finite flux vector is smoothed with a short boxcar kernel,

$$\tilde{F}_i = \frac{1}{N_s} \sum_{j \in \mathcal{W}_i} F_j, \quad (32)$$

where N_s is controlled by the `n_smooth` parameter. This first smoothing is deliberately simple: its purpose is not to define the final continuum, but to stabilize the identification of local maxima and minima.

6.2.3 Step 3: local noise-scale estimate

The spectrum is divided into a configurable number of segments. FitSpec computes the standard deviation in each segment and adopts the median of those values as a robust characteristic local scatter,

$$\sigma_{\text{local}} = \text{median} [\text{std}(F)_1, \dots, \text{std}(F)_{N_s}]. \quad (33)$$

Using a median makes the estimate less sensitive to one segment containing an exceptionally strong spectral feature.

6.2.4 Step 4: local maxima/minima and envelope construction

Relative minima and maxima are detected in the smoothed spectrum. Cubic interpolation through the minima defines a provisional lower envelope,

$$E_{\text{low}}(\lambda), \quad (34)$$

while interpolation through the maxima defines a provisional upper envelope,

$$E_{\text{high}}(\lambda). \quad (35)$$

The difference

$$A(\lambda) = E_{\text{high}}(\lambda) - E_{\text{low}}(\lambda) \quad (36)$$

acts as an *envelope-amplitude spectrum*. Broad emission or absorption structure produces a large separation between the two envelopes, while relatively featureless continuum regions produce a smaller separation.

This step is important because line masking is therefore driven by local spectral morphology rather than by a hard-coded catalogue of transitions.

6.2.5 Step 5: adaptive feature-detection thresholds

FitSpec samples the envelope-amplitude spectrum at a configurable set of positions and estimates a characteristic small-scale scatter. The peak-detection lower threshold is tied to this empirical noise scale, while the upper threshold is tied to the maximum absolute envelope amplitude.

The current implementation uses conceptually

$$h_{\text{noise}} = N_{\sigma} \min [\sigma_A(\mathcal{W}_k)], \quad (37)$$

and

$$h_{\text{data}} = f_{\text{data}} \max |A(\lambda)|, \quad (38)$$

where N_{σ} and f_{data} are configurable.

Broad spectral features are then located using the peak-detection machinery in SciPy ([Virtanen et al., 2020](#)). Feature acceptance simultaneously uses height, prominence, and width constraints. The width limits are expressed relative to a configurable minimum feature width, allowing the algorithm to reject isolated pixel noise while retaining astrophysically broad structure.

6.2.6 Step 6: expand the feature masks

For each detected feature, the left and right interpolated widths supplied by the peak finder are used to define a feature interval. FitSpec expands this interval beyond the measured feature width before constructing continuum-point weights. This reduces the probability that line wings are inadvertently treated as continuum.

6.2.7 Step 7: derivative-based continuum-point weights

The local first difference

$$\Delta F_i = F_i - F_{i-1} \quad (39)$$

is used to construct an inverse-gradient-like weight,

$$w_i \propto \left| \frac{1}{\Delta F_i} \right|. \quad (40)$$

Pixels whose flux changes slowly therefore receive large weights and are preferentially interpreted as continuum-like. Pixels inside the expanded feature intervals are strongly downweighted. In the present implementation the feature-region weight is explicitly reduced by a factor of order 10^{-4} relative to normal weighting.

This stage is one of the distinctive features of the FitSpec continuum method: continuum-point selection is based jointly on detected broad features and local spectral flatness.

6.2.8 Step 8: percentile selection

Only pixels above a configurable percentile of the weight distribution are retained as continuum anchors. If the retained set contains enough points, FitSpec linearly interpolates through them to construct a dense intermediate wavelength sampling.

If too few points survive, a fallback procedure uses the absolute first-difference distribution directly. Pixels with

$$|\Delta F_i| < P_q(|\Delta F|) \quad (41)$$

are retained, where P_q is a configurable percentile. This ensures that the method can still return a continuum for spectra in which the envelope/peak machinery identifies almost the entire spectrum as structured.

6.2.9 Step 9: Savitzky–Golay smoothing

The retained continuum anchors are smoothed using a Savitzky–Golay filter ([Savitzky & Golay, 1964](#)). The Savitzky–Golay method fits low-order polynomials to moving windows using least squares, preserving broad spectral curvature more effectively than a simple moving average for an appropriately chosen window and polynomial order.

FitSpec automatically enforces an odd filter window and ensures that the window remains compatible with the number of available continuum anchors.

For anchor values y_j , the operation may be viewed locally as fitting

$$y(x) = \sum_{k=0}^p a_k x^k \quad (42)$$

within each moving window and evaluating the fitted polynomial at the window centre.

6.2.10 Step 10: continuous continuum interpolation

The Savitzky–Golay-smoothed anchor sequence is finally converted into a continuous wavelength-dependent continuum using cubic interpolation,

$$C(\lambda) = \mathcal{I}_{\text{cubic}} [\lambda_{\text{anchor}}, F_{\text{SG}}]. \quad (43)$$

Because the interpolation is represented as a callable function, the continuum can be evaluated on the native spectrum, a permanently rebinned spectrum, or a GUI display grid without recomputing the anchor selection.

6.2.11 Step 11: continuum-scatter diagnostic

The custom method also reports a characteristic residual scatter around the continuum anchors,

$$\sigma_C = \text{std} [F(\lambda_{\text{anchor}}) - C(\lambda_{\text{anchor}})]. \quad (44)$$

This quantity is useful as a diagnostic of continuum stability, but it should not automatically replace the observational flux uncertainty.

6.3 Why the custom algorithm is different from a simple polynomial

A global polynomial assumes that continuum shape can be represented by one low-order analytic function over the fitted wavelength range. This may be adequate over short intervals, but it becomes problematic when the spectrum contains broad stellar features, strong line complexes, instrument joins, or large-scale curvature.

The FitSpec custom method instead constructs continuum anchors from the morphology of the spectrum itself. Its broad-feature detection removes structured regions before the final smooth representation is formed. Consequently, the method can adapt to spectra in which the continuum is not globally polynomial.

6.4 Alternative continuum estimators

FitSpec intentionally retains multiple continuum estimators because no single method is optimal for all spectroscopic use cases. The methods share one dispatcher and output representation, allowing direct comparison without changing downstream line-fitting code.

6.4.1 Polynomial continuum

The `poly` method fits a polynomial of configurable degree,

$$C(\lambda) = \sum_{k=0}^p a_k \lambda^k. \quad (45)$$

This is computationally inexpensive and useful over narrow wavelength intervals, but can become unstable or physically inappropriate across very wide ranges or spectra dominated by broad features.

6.4.2 Cubic interpolation mode

The current `spline` implementation uses cubic interpolation through the supplied spectral samples. Strictly speaking, this is an interpolating cubic curve rather than a smoothing spline. Because it passes through the supplied samples, it should not be interpreted as a robust noise-rejecting continuum estimator. FitSpec documentation therefore distinguishes this implementation from a penalized smoothing spline.

6.4.3 LOWESS

The `lowess` method uses locally weighted regression following the LOWESS family of smoothers introduced by [Cleveland \(1979\)](#). At each wavelength, nearby samples receive greater weight than distant samples. The configurable fraction determines the local neighbourhood size and therefore the trade-off between following local curvature and suppressing small-scale structure.

LOWESS is useful when the continuum varies smoothly but cannot be represented well by a single global polynomial.

6.4.4 Local Gaussian continuum

The `gauss` method scans the spectrum with a moving window. Within each window, FitSpec fits

$$f(\lambda) = A \exp \left[- \left(\frac{\lambda - \mu}{\sigma} \right)^2 \right] + C_0. \quad (46)$$

The fitted value at the central wavelength becomes the local continuum estimate. Failed local fits are temporarily assigned NaN and subsequently interpolated from successful neighbouring estimates.

This estimator is experimental: a Gaussian plus constant is not a general physical continuum model, but the moving fit can be useful for testing local-background behaviour.

6.4.5 Peak/trough interpolation

The `peak_find` method independently detects peaks in F and troughs in $-F$. The detected extrema and the two spectrum endpoints are collected as anchor positions, and linear interpolation through their fluxes defines the returned curve.

Because spectral features themselves become interpolation anchors, this method is best regarded as an experimental morphology-tracking baseline rather than the default continuum estimator.

6.4.6 Median filtering

The `median_filtering` method evaluates a running median in a configurable wavelength-pixel window,

$$C_i = \text{median}(F_{i-k}, \dots, F_{i+k}). \quad (47)$$

Median filtering is robust against isolated outliers and narrow features when the window is much wider than the contaminating structure. Any undefined values in the resulting continuum are interpolated from valid neighbours.

6.4.7 Friends-of-friends / DBSCAN mode

The `fof` method constructs a wavelength-separation distance matrix and applies DBSCAN, the density-based clustering algorithm introduced by Ester et al. (1996). The current implementation then takes the median flux of one selected wavelength cluster and returns this as a *single flat continuum level* across the spectrum.

This mode is therefore fundamentally different from the wavelength-dependent estimators above. It is retained as an experimental option and should not be confused with a general continuum-shape reconstruction. Because the implementation explicitly constructs a full pairwise distance matrix, its memory requirement grows approximately as N^2 with the number of spectral pixels.

6.4.8 PCA mode

The current `pca` method reshapes one spectrum into a matrix with a single feature and applies a one-component PCA transform followed by the inverse transform. Mathematically, a one-component PCA applied to a one-feature dataset cannot remove a component, so the reconstructed spectrum is effectively the input spectrum itself. FitSpec therefore treats the present implementation as a known no-op rather than presenting it as a scientifically meaningful continuum estimator.

A future PCA continuum method would require a genuine training basis or multi-spectrum feature space. Until such a basis is introduced, this method should remain clearly flagged as experimental/deprecated.

6.5 Interaction with masking

Continuum estimation and the FitSpec science mask are related but distinct. The continuum algorithm may reject features internally when selecting its anchors, but this internal selection does not silently alter the saved stellar/emission/absorption fitting mask.

Conversely, a user may choose to estimate a continuum only from an already selected subset of the spectrum. The key principle is that continuum-point selection and science-pixel selection remain explicit states rather than being conflated.

6.6 Interaction with rebinning and smoothing

Permanent rebinning occurs before continuum estimation if the user explicitly requests it. In that case, the continuum is estimated on the permanently rebinned science spectrum and all saved downstream products use that grid.

GUI smoothing is different. Display smoothing can be applied to the plotted spectrum and continuum for visualization, but it must not change the continuum values used by a scientific fit unless the user explicitly requests a permanent operation.

6.7 Manual continuum interaction

Automatic continuum estimation is not assumed to be infallible. The GUI should allow continuum anchor regions or points to be inspected and, where appropriate, manually adjusted. Manual edits should modify the continuum state rather than changing the underlying observed flux.

The final continuum used for line fitting should therefore be saveable and reloadable in the same spirit as the FitSpec mask state, ensuring reproducibility between an interactive GUI session and a later terminal analysis.

6.8 Continuum uncertainty and future development

The current continuum algorithms primarily estimate a continuum *shape*. The scatter of selected continuum anchors can be reported as a diagnostic, but rigorous propagation of continuum uncertainty into emission- and absorption-line posteriors is a separate inference problem.

Future versions may therefore represent the continuum as

$$C(\lambda; \phi), \tag{48}$$

with continuum parameters ϕ jointly sampled with line parameters. The present modular design deliberately keeps this possibility open.

6.9 What is new in the FitSpec continuum approach

The new contribution is the combination of:

1. one reusable continuum interface for all line-fitting modules;
2. a custom continuum finder that identifies spectral structure from an upper–lower envelope separation rather than relying on a hard-coded line list;
3. explicit broad-feature rejection using peak width, prominence, and adaptive thresholds;
4. derivative-based continuum-point weighting;

5. percentile-based robust anchor selection with a fallback path for difficult spectra;
6. Savitzky–Golay regularization of the selected anchors;
7. continuous interpolation of the final continuum;
8. multiple alternative continuum algorithms exposed through the same interface;
9. clear separation between continuum state, science masks, permanent rebinning, and visualization-only smoothing.

The constituent numerical techniques are established methods, but their assembly into this spectroscopy-oriented workflow is specific to FitSpec.

7 Generic Inference Framework

The `inference/` package is FitSpec’s shared numerical inference layer. It does not define stellar populations, nebular lines, or absorption systems. Those physical models remain in their science modules. The inference layer instead supplies common priors, normalized likelihoods, deterministic/model-selection utilities, posterior problem definitions, sampling backends, posterior products, and diagnostics.

The central design rule is that posterior sampling must not depend on mutable GUI state. A deterministic fit may be performed interactively, but a posterior calculation must be reconstructable later from saved FitSpec products, the physical model definition, and configuration alone.

7.1 Implemented package structure

The inference implementation is divided into the following files:

- `inference/deterministic.py`: compatibility access to the canonical deterministic fitter in `core.fitting`;
- `inference/model_selection.py`: generic ranking and explicit candidate-model search;
- `inference/priors.py`: proper scalar prior distributions and ordered prior sets;
- `inference/problem.py`: sampler-independent `PosteriorProblem`, Gaussian likelihoods, covariance support, and adapters for `ModelParameters`;
- `inference/emcee.py`: optional emcee ensemble-MCMC backend;
- `inference/dynesty.py`: optional static or dynamic dynesty nested-sampling backend;
- `core/results.py`: the shared `PosteriorResult` product, including percentile summaries and compressed portable save/load support;
- `plotting/diagnostics.py`: residual diagnostics plus posterior marginal and corner-style plots.

The optional external sampling packages are deliberately imported only when their engines are run. FitSpec therefore remains usable for deterministic fitting when emcee or dynesty is not installed. Posterior sampling requires the corresponding optional package.

7.2 Deterministic inference

The canonical deterministic implementation remains `core.fitting.fit_deterministic`. The `inference.deterministic` module re-exports this function rather than maintaining a duplicate optimizer. For component-based problems the deterministic fitter operates on an explicit `ModelParameters` object and a model callable,

```
model_func(wave, model_parameters) -> model_flux
```

and returns a `FitResult` containing the fitted parameter state, exact spectrum arrays, working mask, model, statistics, redshift, resolution provenance, and metadata.

A generic wrapper must not erase specialized mathematical structure. The unified stellar fitter remains the principal example: the nonlinear attenuation and kinematic parameters are optimized while non-negative stellar-population and gas-template coefficients are solved internally by bounded linear least squares at every nonlinear trial. This variable-projection algorithm remains part of the stellar science module rather than being replaced by a generic `curve_fit` call.

7.3 Posterior problem abstraction

All posterior engines consume a common `PosteriorProblem`. It contains

- an ordered list of parameter names;
- a log-likelihood callable;
- an ordered `PriorSet`;
- an optional deterministic initial/reference position;
- metadata needed for provenance.

This abstraction is intentionally more general than `ModelParameters`. Emission and absorption models can be adapted directly with `model_parameters_problem`; a specialized fitter may instead provide any physically correct log-likelihood callable with the same flat-vector interface.

For a component model the common construction is

```
from inference import model_parameters_problem

problem = model_parameters_problem(
    wave, flux, flux_unc,
    fit_result.parameters,
    model_func,
    mask=fit_result.mask,
    priors=priors, # optional if every bound is finite
)
```

The adapter copies the supplied `ModelParameters`. Posterior sampling therefore cannot mutate the deterministic result or live GUI parameter state.

A previously saved generic `FitResult` can be used through `problem_from_fit_result`. The science module must reconstruct the correct `model_func` from its physical context, such as the line list, redshift, instrumental resolution, partial-covering choice, and parameter-tying mode. This separation is intentional: a numerical result does not by itself define the physics that generated it.

7.4 Likelihood and covariance handling

For independent Gaussian uncertainties the normalized likelihood is

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{2} \sum_i \left[\frac{(F_i - M_i(\boldsymbol{\theta}))^2}{\sigma_i^2} + \ln(2\pi\sigma_i^2) \right]. \quad (49)$$

The normalization term is retained. It is required for evidence calculations and prevents the absolute likelihood scale from depending on omitted constants.

When a covariance matrix $\mathbf{C}$ is supplied, FitSpec uses

$$\ln \mathcal{L}(\boldsymbol{\theta}) = -\frac{1}{2} \left[\mathbf{r}^T \mathbf{C}^{-1} \mathbf{r} + \ln |\mathbf{C}| + N \ln(2\pi) \right], \quad (50)$$

where $\mathbf{r} = \mathbf{F} - \mathbf{M}$. A full-spectrum covariance matrix is automatically restricted to the active FitSpec mask. Invalid, singular, or non-positive uncertainty definitions are rejected rather than silently repaired.

7.5 Priors

Posterior inference requires an explicit, proper prior measure. FitSpec currently implements

- `UniformPrior`;
- `LogUniformPrior`;
- `GaussianPrior`;
- `TruncatedGaussianPrior`;
- `PriorSet`, which preserves the parameter ordering used by the samplers.

If every free `Parameter` has finite lower and upper bounds, `PriorSet.from_model_parameters` converts those bounds into independent uniform priors. Infinite deterministic bounds are not silently converted into an improper posterior prior. For example, the default emission-line amplitude can be unbounded above; a posterior run must therefore either configure a physically meaningful finite amplitude limit or supply an explicit proper prior.

This distinction is important. Optimizer bounds answer “where may the optimizer search?” while Bayesian priors answer “what probability measure do we assign before seeing these data?” The two may be identical for a bounded uniform prior, but they are not conceptually interchangeable in general.

7.6 emcee posterior sampling

The emcee backend is called with

```
from inference import run_emcee

posterior = run_emcee(
    problem,
    nwalkers=64,
    nsteps=4000,
    burn=1000,
    thin=5,
```

```

    random_seed=1234,
    deterministic_result=fit_result,
)

```

Walkers are initialized in a parameter-scale-aware cloud around the deterministic reference solution when one is available. Any invalid initial walker is replaced by an independent prior draw. If there is no valid reference solution, all walkers are drawn from the prior.

The returned `PosteriorResult` contains the flattened post-burn-in samples and the corresponding log posterior probabilities. Its metadata records the number of walkers and steps, burn-in and thinning, random seed, individual and mean acceptance fractions, and an integrated autocorrelation-time estimate when emcee can determine one. A chain should not be interpreted solely because the sampler terminated; acceptance behavior, autocorrelation length, trace stability, and the scientific posterior structure must be inspected.

7.7 dynesty nested sampling

Dynesty is useful when multimodality or Bayesian evidence is scientifically important. FitSpec supports both static and dynamic nested sampling:

```

from inference import run_dynesty

posterior = run_dynesty(
    problem,
    dynamic=True,
    nlive=500,
    dlogz=0.1,
    random_seed=1234,
    deterministic_result=fit_result,
)

```

The `PosteriorProblem.prior_transform` maps the unit cube into the physical priors, while dynesty receives the likelihood separately. The native weighted nested samples are converted to equal-weight posterior samples for the common `PosteriorResult`. Metadata retains the final log evidence, its reported uncertainty, the number of likelihood calls, live-point setting, stopping tolerance, and sampling/bounding choices.

For comparing genuinely distinct physical models using evidence, the priors must be scientifically defensible and consistently defined. Bayes factors are prior-sensitive and should not be treated as an automatic replacement for physical judgment.

7.8 Posterior result products

`PosteriorResult` is the common output of both sampling engines. It provides

- `samples`: shape $(N_{\text{sample}}, N_{\text{par}})$;
- `log_probability`: one value per stored sample;
- `ordered_parameter_names`;
- an optional linked deterministic `FitResult`;
- `sampler/provenance` metadata;

- `percentiles()` and named `summary()` methods;
- `maximum_posterior_sample`;
- `save_npz()` and `load_npz()` for compressed, non-pickle posterior products.

The standard 16th, 50th, and 84th percentiles provide convenient one-dimensional credible summaries, but FitSpec retains the full posterior samples because asymmetric, correlated, or multimodal distributions cannot be represented faithfully by one central value and one symmetric uncertainty.

7.9 Posterior diagnostics

`plotting/diagnostics.py` now provides `plot_posterior_marginals` and `plot_posterior_corner`. The latter is intentionally implemented with NumPy/Matplotlib only, avoiding a mandatory dependency on the external `corner` package. For high-dimensional fits it limits the default display to the first eight parameters; the caller may request a scientifically relevant subset explicitly.

These plots diagnose marginal shape and pairwise covariance. They do not by themselves establish MCMC convergence. emcee convergence information is stored separately in the posterior metadata.

7.10 Model selection

Model-selection statistics remain defined in `core.statistics`. The inference layer adds the generic driver rather than duplicating the formulae. `compare_models` ranks already-fitted candidates by BIC, AIC, AICc, chi-square, or reduced chi-square, while `select_best_model` calls a science-provided fitting function for a list of explicit candidate values.

The component number remains explicit application state. For example,

```
best, ranking = select_best_model(
    [1, 2, 3, 4],
    lambda n: fit_with_n_components(n),
    criterion="bic",
)
```

The generic driver never infers component count from a scalar/list datatype or from the length of an unrelated configuration value.

7.11 Relationship to the science modules

Emission and absorption fitting use the shared `ModelParameters` representation and are therefore directly compatible with `model_parameters_problem`. Both modules now provide complete science-module adapters (`emission/inference.py` and `absorption/inference.py`) that convert an accepted deterministic result into a `PosteriorProblem` and dispatch it to emcee or dynesty from either the terminal or the corresponding standalone GUI. Their fixed parameters remain excluded from the sampled vector exactly as they are excluded from deterministic optimization, and physical ties continue to be applied at every likelihood evaluation. The posterior run conditions on the component structure accepted by the deterministic stage; component-number comparison or absorption rejection remains an explicit model-construction/model-selection operation rather than being hidden inside parameter sampling.

The two modules differ only where their physics requires it. Emission amplitudes may be unbounded above during deterministic optimization, so emission inference requires an explicitly finite amplitude prior. Standard absorption parameters already carry finite physical bounds, and those bounds become the default uniform priors. For joint absorption fits, generic cross-ion ties, thermal/turbulent links, and abundance-pattern constraints are reapplied inside the posterior model. Components frozen by absorption’s rejection stage remain fixed and retain their upper-limit/non-detection interpretation; posterior sampling does not turn them back into independent detections. Arbitrary Python tie transforms are not serialized as executable code, so a joint posterior run must be supplied the same code-level tie objects that defined the deterministic model.

The stellar fitter is deliberately different. Its deterministic algorithm uses variable projection: for each active nonlinear trial parameter (by default only attenuation; stellar velocity and dispersion are fixed unless explicitly enabled, and nuisance gas is off by default), the non-negative stellar-population and optional emission-template coefficients are re-solved conditionally. A posterior that merely samples the nonlinear parameters while re-optimizing the linear coefficients is a *profile likelihood*, not a marginal posterior over stellar-population weights. FitSpec therefore exposes this calculation only as the explicitly named `profile` stellar-inference mode and records `profile_not_marginalized` in the posterior metadata.

The implemented stellar-specific adapter in `stellar/inference.py` also provides the recommended `conditional` posterior, which samples the nonlinear parameters and selected non-negative SSP amplitudes together. SSP masses are parameterized as $\log_{10}(M/M_{\odot})$ with finite uniform bounds, corresponding to proper log-uniform mass priors. The sampled SSP basis can be the deterministic active set, a broader diagnostic candidate set, or the entire loaded library. Candidate mode is the default: it retains all active SSPs and adds low- $\Delta\chi^2$ alternatives identified by the deterministic single-SSP landscape, up to a configurable dimensionality cap. This posterior-basis choice is independent of whether the deterministic fit itself used `stellar_template_selection=candidate` or `full`. This allows age-metallicity alternatives with zero deterministic NNLS weight to acquire posterior support without forcing hundreds of templates into every sampling run.

The resulting conditional calculation is a full posterior over the parameters included in that chosen basis; it remains conditional on the basis-selection rule itself. Full-library mode is available for intentionally small/restricted libraries and is protected by a maximum-template guard. Optional stellar nuisance-gas amplitudes may likewise be sampled for the deterministic active gas set or for all available nuisance templates. Thus FitSpec makes the conditioning assumptions explicit rather than disguising variable projection as marginalization.

7.12 Recommended terminal workflow

The intended reproducible workflow is

1. perform and inspect a deterministic fit in the GUI or terminal;
2. save the deterministic product and the physical model/configuration needed to rebuild the model callable;
3. construct a proper `PriorSet`;
4. create a `PosteriorProblem` from the saved result or science-specific likelihood;
5. run emcee for posterior exploration or dynesty when nested sampling/evidence is required;
6. inspect convergence and posterior diagnostics;

7. save the independent posterior product with `PosteriorResult.save_npz`;
8. report credible intervals or model evidence only after confirming that priors, degeneracies, masks, and convergence are scientifically acceptable.

No posterior run requires an open FitSpec GUI, and no sampler is allowed to read mutable widget state.

For emission specifically, the supported module-level workflow is

```
from emission import fit_emission_spectrum, run_emission_inference

fit = fit_emission_spectrum(spectrum, config)
posterior = run_emission_inference(fit, spectrum, config)
```

where `emission_inference_method` selects `emcee`, `dynesty`, or leaves posterior sampling disabled with `deterministic`. Because default emission amplitudes are unbounded above for deterministic optimization, a posterior run must define a finite proper amplitude prior through either `emission_maximum_amplitude` or `emission_inference_amplitude_prior_max`.

For absorption, a single-group posterior uses

```
from absorption import fit_absorption_spectrum, run_absorption_inference

fit = fit_absorption_spectrum(spectrum, config, transitions=transitions)
posterior = run_absorption_inference(fit, spectrum, config, transitions=transitions)
```

where `absorption_inference_method` selects `emcee`, `dynesty`, or leaves sampling disabled. Joint cross-ion posteriors use the same function with `systems=` and, when applicable, the original `ties=`, `thermal_turbulent_links=`, and `abundance_pattern_groups=` objects. This preserves the deterministic model's physical linkage and its fixed/rejected component state exactly.

For stellar populations, the corresponding API is

```
posterior = run_stellar_inference(
    stellar_result, spectrum, config, library=library
)
```

where `stellar_inference_posterior_mode=conditional` samples the nonlinear parameters together with the selected SSP masses, while `profile` intentionally performs the faster non-marginalized variable-projection posterior. The basis-selection policy is controlled independently by `stellar_inference_basis_m`. A stellar posterior requires the same loaded stellar library and observational resolution model used to construct the deterministic fit, because those objects are needed to regenerate transformed SSP spectra continuously during sampling.

8 GUI Architecture

FitSpec should have one common GUI shell and three science-specific panels rather than three unrelated GUI applications.

The common GUI should provide functionality such as:

- loading spectra;
- zooming and navigation;
- masking and unmasking;

- continuum interaction;
- display-only smoothing;
- saving/loading sessions;
- launching deterministic fits;
- plotting residuals and fit regions;
- saving fit results.

Science-specific controls should remain in specialized stellar, emission, and absorption panels.

8.1 Explicit Component Controller

For emission and absorption fitting, components should be represented explicitly. A model can own a list of component objects, for example:

```
components = [
    Component(...),
    Component(...),
    Component(...),
]
```

The GUI should provide two central controls:

1. **Number of components:** the number currently available in the model;
2. **Active component:** the component currently being edited and highlighted.

Amplitude or column density, velocity, and velocity dispersion controls should update only the active component while preserving the most recent values of all other components. Each component should retain its own parameter state even when inactive.

Only the active component needs to be visually emphasized separately from the total model, while the total model should always reflect all components.

The component count must be an explicit setting. Configuration arrays may initialize parameters, but array length should not silently determine the physical model.

8.2 Implementation

The design requirements above are implemented as a set of reusable, Matplotlib-widget-based controllers (sliders, buttons, and radio buttons embedded directly in a **Figure** – no separate GUI framework dependency) shared across three science-specific panel classes, plus a common application shell. The principal GUI modules are `gui.state`, `gui.base_gui`, `gui.app`, `gui.mask_controller`, `gui.component_controller`, and the science panels `gui.stellar`/`gui.emission`/`gui.absorption` (mirroring `plotting.stellar`/`plotting.emission`/`plotting.absorption`, Section 10.1). The user can therefore launch one FitSpec application, load one spectrum and one run configuration, and open any of the three science panels without creating independent copies of the spectrum or session state.

8.2.1 Common application shell and shared session state

`gui.state.SessionState` is the target-level state container shared by the application and every science panel. It owns the single universal `Spectrum` object, its source path and run directory, the three layered mode configurations, the most recent deterministic and posterior result for each science mode, and references to any currently open panel. Passing the same `Spectrum` object by identity is intentional: a mask edit or other target-level state change is therefore visible to all panels rather than being copied into three diverging GUI sessions. Replacing the spectrum explicitly invalidates all previous fit, posterior, and panel state so results from one target cannot be accidentally reused for another.

The portable session serializer, `SessionState.save`, writes the common spectrum arrays, uncertainty, continuum/model arrays when present, current boolean mask, redshift, and JSON-safe provenance to a compressed `fitspec_session.npz`. It deliberately does *not* pickle live Python result or configuration objects. Deterministic fits and posteriors retain their science-module FITS/NPZ serializers, while a restored session rebuilds configuration objects from the authoritative default files plus the current `config.dat`. This avoids stale executable state and keeps the session format inspectable.

`gui.base_gui.BaseGUI` owns the common shell figure: a target-level spectrum view, masked-pixel overlay, session status, mode-launch controls, and session saving. `gui.app.FitSpecApp` specializes that shell by lazily constructing `StellarGUI`, `EmissionGUI`, or `AbsorptionGUI` on request. Each panel receives the shared `SessionState`, publishes deterministic/posterior results back into it, and remains responsible only for its science-specific controls. Lazy panel imports keep application startup independent of libraries needed only by one science mode.

The executable `fitspec.py` is the user-facing entry point. A typical launch is

```
python fitspec.py spectrum.dat --run-dir ./my_run
```

where `./my_run/config.dat`, when present, overlays the base and mode-specific defaults for all three panels. The unified loader validates that file against the union of the stellar, emission, and absorption schemas, then applies each recognized keyword only to the mode(s) that define it; a genuinely unknown keyword remains a fatal configuration error. The shipped configuration files are self-documenting schemas: optional recognized keywords remain visible as commented `key = example` entries and can be uncommented in `config.dat`. Nested interval syntax such as `excluded_intervals=[[900,1100],[1140,1147]]` is parsed natively.

The launcher settings `input_spectrum`, `redshift`, and `mode` may also be stored in `config.dat`, so a target run directory can be launched with `fitspec` alone. Relative spectrum paths are resolved relative to the run/config directory. Explicit command-line values retain highest precedence: CLI > `config.dat` > defaults. `-session fitspec_session.npz` restores a previously saved target session. The command-line shell and direct Python API both route through the same `build_session` and `FitSpecApp` classes.

8.2.2 Mask controller and the three-tier mask model

Interactive masking is implemented as a three-tier state, `core.masking.FitMaskState`, wrapped for GUI use by `gui.mask_controller.MaskController`:

- **initial_mask** – built once from the configuration-driven region/velocity-window selection (Section 9) when the controller is constructed, and never changed afterward by GUI interaction.

- **working_mask** – the mask actually used for the live preview and any fit triggered from the GUI. Every interactive edit – rectangle SET/UNSET on the spectrum axes, or a velocity-window width change – modifies only this tier, immediately.
- **saved_mask** – the last explicitly committed state. `Save Mask` copies `working_mask` into `saved_mask` and writes it to a per-spectrum compressed `.npz` file (`core.masking.save_mask_file`); `Load Mask` does the reverse, discarding any uncommitted edits in `working_mask`. Loading a saved mask checks that the stored wavelength grid matches the current spectrum's (`core.masking.load_mask_file`) and refuses a mismatched file outright, so a mask cannot be silently reapplied after a different permanent rebinning (Section 9) changed the grid it was built for.

At every tier, invalid pixels (non-finite flux, or a non-finite/non-positive uncertainty) can never be reactivated – `FitMaskState` enforces this unconditionally on construction and after every edit, the same invariant `core.masking.combine_masks` enforces for the non-interactive path. Rectangle SET/UNSET operates in both wavelength and flux coordinates at once (a click-and-drag selects a 2-D region of the plotted data, not just a wavelength range), and velocity-window-width changes are available only when `fit_mode` is "emission" or "absorption" – stellar fitting has no line-centered velocity mask and the corresponding controls are simply absent for it, rather than present but inert.

`MaskController.connect_matplotlib` takes only axes objects from the caller (never creating its own subplot layout), so the same controller is reusable verbatim across all three science panels while each panel retains full control of its own figure layout.

8.2.3 Explicit component controller

`gui.component_controller.ComponentController` is the Matplotlib-widget layer on top of `core.parameters.ModelParameters` which already implements the component list itself – explicit `n_components`, `add_component/remove_component`, and an `active_component_index` that changing never mutates (Section 3) – so the controller's job is purely to wire that existing model-layer logic to widgets, not to reimplement it. Constructed with a `ModelParameters` instance, a list of *controlled parameter names* (e.g. ["velocity_kms", "sigma_kms"] for emission, ["logN", "b_kms", "velocity_kms"] for absorption), and a `make_new_component` factory (needed because a new component's sensible initial values/bounds are science-module-specific and cannot be inferred generically), it provides:

- **Component count:** +/- buttons that call `add_component/remove_component` directly, refusing to remove the last remaining component.
- **Active component:** a `RadioButtons` selector (rebuilt whenever the count changes, since Matplotlib's `RadioButtons` cannot have its label set changed in place) that calls `set_active`.
- **Per-parameter sliders:** one `Slider` per controlled parameter name, always displaying and editing only the *active* component's value. Switching the active component updates every slider's *displayed* value to that component's stored value with slider callbacks suppressed during the update (so the resync itself never triggers a spurious edit), then releases the suppression – guaranteeing the documented invariant that switching components never alters any component's values, and that a slider drag never affects any component but the active one.

Emission fitting has a per-line integrated-flux amplitude rather than one generic "amplitude" per component – a fitted line list can have many free lines, which does not fit a fixed slider-per-parameter

layout. `gui.emission.EmissionGUI` therefore adds an independent, second selector (which *line* is active) alongside `ComponentController`'s (which *component* is active), with a single amplitude slider editing the selected component's selected line – the same "switching selection never alters other values" invariant, applied along a second, orthogonal axis, rather than one slider per line.

8.2.4 Stellar, emission, and absorption panels

`gui.stellar.StellarGUI` previews the selected SSP template against the data and calls the single unified `fit_stellar_spectrum` routine on demand (Section 5); having no explicit multi-component list, it does not use `ComponentController`. The stellar panel exposes age, metallicity, attenuation, stellar-velocity, stellar-dispersion, and display-bin controls in separated compact slider groups so their labels do not overlap. Stellar velocity and dispersion are independently opt-in fit parameters and are fixed by default; nuisance gas is also off by default. GUI actions print terminal feedback, including fit settings and live optimizer improvement. The initial wavelength and flux axes are derived from the loaded spectrum rather than from empty Matplotlib defaults.

The deterministic stellar Plot action saves the full established product set—main result, observational summary, six-panel diagnostics, and diagnostics text—using version-neutral filenames. Save writes the deterministic fit and its sidecar products. Posterior controls remain separate (Posterior/Load P/Save P/Plot P). Save Session belongs to the common application shell and stores portable target/session state; it is not a replacement for saving the science fit.

`gui.emission.EmissionGUI` and `gui.absorption.AbsorptionGUI` follow the same layout pattern: a spectrum axes showing data, masked points (dimmed, not dropped), a live model preview, and the best fit once run; a display-smoothing slider (Section 9, GUI-only, never touching the stored spectrum); `ComponentController` wired to that module's per-component parameters; `MaskController` in the corresponding `fit_mode`; and Fit/Save Fit/Load Fit buttons calling `fit_emission_spectrum/fit_absorption_spectrum` and the corresponding `save__result/load__result` functions directly. `AbsorptionGUI` additionally exposes a covering-fraction slider, present only when `absorption_partial_coverage` is enabled (Section 5.21.5), bound to component 0 – the single free holder of that shared parameter – exactly matching how the fitting code itself treats it; a newly-added component's covering-fraction copy is created already fixed, consistent with `build_absorption_parameters`.

Cross-ion joint fitting, thermal/turbulent Doppler linking, abundance-pattern ties, region shifts, in-fit continuum adjustment, and automatic component rejection (Section 5.22) are, by their own design, code-level constructs built from explicit component-index references rather than GUI-editable settings, and are intentionally not exposed through `AbsorptionGUI`'s single-transition-group interactive session; a fit built programmatically with those features is inspected afterward with `plotting.absorption` instead.

9 Rebinning, Smoothing, Resolution, and Masking

9.1 Permanent Rebinning Versus Display Smoothing

FitSpec uses one flux-conservative, gap-aware rebinning implementation, `core.rebinning.rebin_spectrum`. The distinction between permanent rebinning and smoothing is a distinction in application state, not in numerical method.

The numerical core builds a sparse wavelength-overlap operator A (`flux_bin = A @ flux`) and propagates the full covariance through the same operator ($C_{\text{bin}} = A C A^T$, falling back to a diagonal covariance from `flux_unc` if none is supplied). Gaps are handled by two independent, combinable

controls: `min_coverage` drops (NaNs) an output bin if too little of its wavelength range is covered by valid input pixels, propagating gaps honestly rather than interpolating over them; `max_gap_pixels` caps how many consecutive missing native pixels may be bridged when gap-filling interpolation is explicitly requested (`fill_gaps=True`), and never extrapolates past the valid-data edges.

Permanent rebinning (`apply_permanent_rebinning`) occurs only when requested in configuration and is performed immediately after loading/validation, before fitting; it overwrites the stored `Spectrum` (including resampling its boolean mask through the same operator: a bin is masked if any contributing input pixel was masked). The rebinned spectrum becomes the spectrum used by all subsequent fits and saved products.

Display smoothing (`compute_display_smoothing`) is temporary and GUI-only: the same flux-conservative algorithm, evaluated at a (typically coarser) bin size, returning display-only arrays without touching the stored spectrum, uncertainties, masks, or fit calculations. Covariance propagation is skipped by default for this path, since only a display uncertainty envelope is needed.

9.2 Spectral Resolution and Instrumental Convolution

Resolution enters a fit in two genuinely different ways, and `core.resolution` handles them with two different mechanisms rather than forcing both through one numerical convolution:

- **Single-line models** (a Gaussian or Voigt line profile) need no numerical convolution at all. Convolving a Gaussian with an instrumental Gaussian is again a Gaussian, and since a Voigt profile is already $\text{Gaussian} * \text{Lorentzian}$, convolving a Voigt line with an instrumental Gaussian is exactly equivalent to using a larger *effective Doppler parameter* and leaving the Lorentzian damping untouched – this is the standard parameterization (e.g. PyAstronomy’s `VoigtAstroP`). `combine_gaussian_sigma` and `effective_doppler_b_kms` implement this closed-form combination.
- **Full-spectrum / template models** (e.g. stellar population templates), where resolution genuinely varies across the fitted bandpass, need real numerical convolution with a wavelength-dependent kernel. `convolve_variable_gaussian` implements the standard direct-summation approach (Cappellari’s `ppxf_util.gaussian_filter1d`; Westfall et al. 2019), evaluated directly on the target (data) wavelength grid, which also performs the required resampling onto the observed grid in the same pass rather than as a separate step.

A `ResolutionModel` wraps where the resolution came from – `from_table` (wavelength-dependent $R(\lambda)$, FWHM, or σ , linearly interpolated), `from_constant_fwhm_kms` (a single Gaussian FWHM in km s^{-1} across the whole spectrum), `from_file` (a delimited resolution table), or `default_fallback` – and exposes `sigma_angstrom(wave)`, `fwhm_angstrom(wave)`, and `R(wave)`.

If the user supplies no instrumental resolution for a spectrum, `ResolutionModel.default_fallback` requires an explicit file path (there is no built-in fabricated default table – inventing plausible-looking instrument numbers would itself be a silent correctness bug) and raises `DefaultResolutionWarning` immediately upon construction, not deferred and not swallowable by a caller. The warning states that incorrect instrumental resolution can bias intrinsic velocity dispersions, line widths, column densities, or stellar-population parameters.

9.3 Masking

Internally, all masking mechanisms resolve to a single boolean mask defined on the spectrum wavelength grid, built by `core.masking.combine_masks` in a fixed composition order:

1. The *selection* – which pixels are candidates for fitting at all – comes from **region** (**region_mask**: wavelength intervals from configuration) and/or **velocity** (**velocity_window_mask**: a window around one or more line centers, OR'd across lines), combined with OR. If neither is given, every pixel starts selected.
2. **interactive_add** / **interactive_remove** (GUI add/remove-point editing) then explicitly add or remove individual pixels from that selection.
3. **invalid** (non-finite flux or bad uncertainty, from **core.spectrum.clean_spectrum**) is subtracted **last and unconditionally** – no combination of region, velocity, or interactive settings, including an explicit interactive add, can force an invalid pixel into the fit.

This directly covers the required mask sources: wavelength regions from configuration; velocity limits around spectral lines; invalid or non-finite pixels; GUI region selection and manual add/remove-point operations; and science-specific fit regions (built from the same **region_mask/velocity_window_mask** primitives).

10 Results, Saving, and Plotting

A standardized result object is used across fitting modes. **core.results.FitResult** bundles:

```
FitResult:
  parameters: ModelParameters
  parameter_uncertainties: dict[str, float] | None
  wave, flux, flux_unc: np.ndarray # full arrays actually used
  mask: np.ndarray
  model: np.ndarray # best fit, on the full wave grid
  statistics: FitStatistics # chi2, reduced chi2, BIC, AIC, AICc, ...
  redshift: float
  resolution_source: str | None
  method: str
  metadata: dict
```

Fit statistics (chi-square, BIC, AIC, ...) are nested inside a single **FitStatistics** object (**core.statistics**) rather than duplicated as flat fields, so every science module's fit produces directly comparable statistics from one shared implementation. **residuals** is a derived property (**flux - model**).

Science-specific result classes or extensions may then store additional quantities such as stellar masses and ages, emission-line fluxes, or absorption-line column densities.

Save/load functionality preserves enough information to recreate the fit without relying on the live GUI: **FitResult.to_dict/from_dict** serialize the exact spectrum, mask, best-fitting parameters (via **ModelParameters.to_dict/from_dict**), parameter uncertainties, and model arrays, so posterior sampling can be initialized from a saved deterministic fit alone, from the terminal, without the GUI.

core.results.PosteriorResult is the analogous container for Dynesty/emcee posterior samples (**samples**, **log_probability**, **parameter_names**, and percentile summaries), optionally linked back to the **FitResult** it was seeded from.

Plotting should remain separate from fitting and should receive saved or in-memory result objects rather than running science calculations itself. Plotting modules may include full-spectrum plots, line stamps, residuals, stellar diagnostics, component plots, and posterior diagnostics.

10.1 Plotting

10.1.1 Architecture

Plotting is deliberately kept separate from fitting: every function in `plotting/` receives a result object (`core.results.FitResult`, or a science module's wrapper around one) and never runs a science calculation itself. The module is layered in three parts:

- **Generic primitives** (`plotting.spectrum`, `plotting.residuals`, `plotting.stamps`, `plotting.diagnostics`) operate on any `FitResult` – or on bare wavelength/flux arrays – and are shared by every science module, since stellar, emission, and absorption fits all produce the same `FitResult` shape (Section 3).
- **Per-module wrappers** (`plotting.emission`, `plotting.absorption`, `plotting.stellar`¹) compose those primitives with module-specific context – a fitted line list, per-component kinematics, stellar population diagnostics – rather than duplicating plotting logic, per the "define universal functionality once" design principle.
- `plotting.stamps.plot_velocity_stack` is FitSpec's non-interactive counterpart to RDGEN's signature stacked common-velocity-scale plot (its cursor-mode `v/u` commands, Section 6.1 of the RDGEN manual (Carswell & Webb, 2014)): several transitions or lines, often from different species, plotted on one shared velocity axis relative to a common reference redshift and stacked with a vertical offset, so common kinematic structure across species is visible at a glance. It is duck-typed over "line-like" objects – anything with a `.name` and a `.rest_wavelength_angstrom` – so it works identically for `emission.lines.EmissionLine` and `absorption.atomic.AtomicTransition` without this module importing either science module directly.

10.1.2 Generic primitives

`plotting.spectrum.plot_spectrum_fit` draws the standard data+model(+residual panel) figure for any `FitResult`: masked-out points are shown dimmed rather than dropped, uncertainty is shaded when available, and the residual panel (when requested) shares the wavelength axis. `plot_spectrum_fit_axes` is the composable form, drawing onto axes the caller already created, for building custom multi-panel layouts (e.g. alongside a velocity stack).

`plotting.residuals.add_residual_panel` plots (flux–model), normalized by `flux_unc` when available (so the panel reads directly in units of σ , with $\pm 1\sigma$ reference lines), falling back to a raw residual otherwise.

`plotting.stamps.wavelength_to_velocity_kms` inverts the relativistic Doppler formula used throughout the emission and absorption line-profile code (Sections 5.19, 5.21) to convert an observed wavelength to a velocity offset from a rest wavelength at a given systemic redshift, so plotting and fitting always agree on what "velocity" means. `plot_line_stamps` lays out a grid of small panels, each independently zoomed to $\pm$ a configurable velocity half-width around one line – useful for visually comparing line shapes, widths, and detections across many transitions at once. `plot_velocity_stack` is the stacked plot described above; it optionally overlays a vertical marker at each of a list of component velocities (e.g. an absorption or emission fit's fitted per-component velocities), so the reader can see exactly where the model places each kinematic component relative to every line simultaneously, and optionally normalizes each line's local flux before stacking so lines

¹Historically `plotting/stellar_plotting.py`; both the module and this document now refer to it simply as `plotting.stellar`.

of very different absolute depth remain equally visible (a display-only rescaling that never touches the underlying fitted values).

`plotting.diagnostics.plot_residual_diagnostics` produces a three-panel figure – residuals vs. wavelength, a residual histogram compared against the expected unit Gaussian, and a table of the fit’s `FitStatistics` – as a fast "did this fit go well?" check that needs only a deterministic `FitResult`. This is deliberately scoped to what the current infrastructure provides: once the inference layer (Section 7) produces real posterior samples, corner-plot-style posterior diagnostics belong in this module too, but are not yet implemented.

10.1.3 Emission-fit plotting

`plotting.emission` wraps `EmissionFitResult` (Section 5.19). `plot_emission_fit` is the full-spectrum data+model(+residual) view; `plot_emission_line_stamps` lays out a stamp grid over the fitted line list; `plot_emission_velocity_stack` produces the stacked plot, reading its line subset and velocity range from configuration (`emission_fig_plot_lines`, `emission_fig_velocity_limit_kms`, `emission_fig_y_boost` – see `config/default_config_emission.dat`) when a config object is supplied, and marking every fitted kinematic component’s velocity; `plot_emission_line_fluxes` is a bar chart of every measured line’s integrated flux with its propagated uncertainty (Section 5.19’s tied-ratio uncertainty propagation included).

10.1.4 Absorption-fit plotting

`plotting.absorption` wraps `AbsorptionFitResult` (Section 5.21) with the analogous set of functions – `plot_absorption_fit`, `plot_absorption_line_stamps`, and `plot_absorption_velocity_stack` (reading `absorption_fig_plot_transitions/absorption_fig_velocity_limit_kms` from configuration) – plus `plot_absorption_component_summary`, a two-panel log N -vs-velocity and b -vs-velocity error-bar plot that visually distinguishes ordinary fitted components from ones frozen by the automatic-rejection mechanism (Section 5.22.8): a frozen component’s log N is a upper limit rather than a detection, and is drawn as a downward-pointing marker with no error bar rather than a circle with one, making the distinction impossible to miss at a glance.

10.1.5 Stellar-fit plotting

`plotting.stellar` (developed alongside the unified stellar fitter, Section 5) provides `plot_stellar_fit`, `plot_stellar_diagnostics`, and `plot_stellar_observational_summary`, covering the data/-model/residual view together with light-fraction and degeneracy diagnostics specific to the variable-projection stellar fit.

11 FitSpec Test Run

FitSpec should provide a self-test mode such as:

```
fitspec --test-run
```

The test should check more than simple Python imports. It should verify, as applicable:

1. supported Python version;
2. required core dependencies;
3. optional fitting/sampling dependencies;

4. presence of required FitSpec modules;
5. default configuration files;
6. resolution tables;
7. emission-line and atomic-data files;
8. stellar libraries or library availability;
9. bundled test spectrum;
10. ability to load and validate that spectrum;
11. a small rebinning operation;
12. a model-convolution/resampling check;
13. lightweight stellar, emission, and absorption fitting checks where practical.

The bundled test spectrum may contain representative UV and optical regions so that the installation test exercises the major code paths without requiring a science dataset from the user.

12 Directory Structure

The current package organization is:

```
FitSpec/
|-- README.md
|-- fitspec.py
|-- config/
| |-- default_config.dat
| |-- default_config_stellar.dat
| |-- default_config_emission.dat
| '-- default_config_absorption.dat
|-- core/
| |-- spectrum.py
| |-- io.py
| |-- config.py
| |-- rebinning.py
| |-- resolution.py
| |-- wavelengths.py
| |-- masking.py
| |-- models.py
| |-- parameters.py
| |-- fitting.py
| |-- statistics.py
| '-- results.py
|-- continuum/
| '-- continuum.py
|-- stellar/
| |-- stellar_models.py
| |-- stellar_fit.py
| |-- stellar_results.py
| |-- inference.py
```



```
| '-- __init__.py
|-- emission/
| |-- lines.py
| |-- profiles.py
| |-- emission_model.py
| |-- emission_fit.py
| |-- emission_results.py
| '-- inference.py
|-- absorption/
| |-- atomic.py
| |-- profiles.py
| |-- absorption_model.py
| |-- absorption_fit.py
| |-- absorption_results.py
| |-- inference.py
| |-- rejection.py
| |-- upper_limits.py
| '-- synthetic.py
|-- inference/
| |-- deterministic.py
| |-- problem.py
| |-- priors.py
| |-- model_selection.py
| |-- dynesty.py
| |-- emcee.py
| '-- __init__.py
|-- gui/
| |-- app.py
| |-- base_gui.py
| |-- stellar.py
| |-- emission.py
| |-- absorption.py
| |-- component_controller.py
| |-- mask_controller.py
| '-- state.py
|-- plotting/
| |-- spectrum.py
| |-- stamps.py
| |-- residuals.py
| |-- stellar.py
| |-- emission.py
| |-- absorption.py
| '-- diagnostics.py
|-- data/
| |-- resolution_average.dat
| |-- nebular_emission_lines.dat
| |-- atomic_absorption_lines.dat
| '-- test_spectrum.fits
|-- tests/
| |-- test_io.py
| |-- test_rebinning.py
| |-- test_resolution.py
| |-- test_models.py
| |-- test_stellar.py
```

```
| |-- test_stellar_gui_runtime.py
| |-- test_config_exhaustive.py
| |-- test_emission.py
| |-- test_absorption.py
| |-- test_inference.py
| |-- test_stellar_inference.py
| |-- test_emission_inference.py
| |-- test_absorption_inference.py
| '-- test_app_shell.py
'-- test_run.py
```

The root `README.md` provides the concise user-facing entry point, while `test_run.py` is the lightweight end-to-end smoke test. The exhaustive configuration schemas live under `config/` and include commented optional keys that remain valid when activated in a run-specific `config.dat`.

The stellar directory intentionally contains no separate `ficus.py`, `ppxf.py`, or `sessamme.py` backends. The unified stellar fitting algorithm lives in `stellar_fit.py`; library-specific behavior is isolated in `stellar_models.py`. The GUI and plotting implementations remain outside the science module, in `gui/stellar.py` and `plotting/stellar.py`, respectively.

The `inference/` directory contains the implemented sampler-independent posterior problem, priors, deterministic adapters, model-selection helpers, and emcee/dynesty backends. It does not require every science module to use the same optimizer internally. Science-specific algorithms may remain in their science modules while calling shared likelihood, statistics, model-selection, or posterior-sampling utilities when appropriate.

This structure is intentionally modular but not fragmented. Individual files may later be split if they become genuinely unwieldy; shared functionality should not be duplicated merely because it is used by different science modes, and science-specific algorithms should not be split solely to mimic historical external fitting packages.

13 Design Principles

The following principles should guide future development:

1. **One application, modular implementation.** FitSpec should feel like one program without becoming one monolithic source file.
2. **Define universal functionality once.** Common numerical and data-handling routines should have one implementation with configurable parameters.
3. **Reuse inference without erasing scientific structure.** Likelihoods, statistics, model-selection criteria, priors, and posterior samplers should be shared where their mathematics is common. Specialized science algorithms such as stellar variable projection may remain within their science module rather than being forced into an unsuitable generic optimizer wrapper.
4. **One universal stellar architecture.** UV and optical stellar fitting use the same FitSpec stellar-fitting framework; differences in wavelength regime or stellar library must not recreate separate FICUS, pPXF, SESSAMME, or similar backend architectures.
5. **Explicit state beats implicit inference.** The number of model components should be stored explicitly rather than inferred from whether a configuration parameter happens to be scalar or list-valued.

6. **GUI state should not contain science logic.** GUI widgets manipulate model/application state, but scientific calculations should live in reusable modules callable without a GUI.
7. **Posterior sampling should be reproducible without the GUI.** Saved deterministic results must carry enough information to initialize Dynesty or emcee from the terminal.
8. **Permanent transformations must be distinguished from visualization.** Permanent configured rebinning changes the analysis spectrum; display smoothing does not.
9. **Resolution must never be silently ignored.** Every fitted model must account for instrumental resolution. Default resolution fallback must produce a prominent warning.
10. **Masking should have one authoritative working representation.** Configuration intervals establish the initial selection; GUI operations modify the working mask; velocity-window operations are available only to emission and absorption fitting; explicit Save Mask and Load Mask actions control persistence.
11. **Plotting should be downstream of fitting.** Plotting should consume results, not determine or modify the fit.

This architecture is intended to allow FitSpec to grow without locking future development to the assumptions of the original independent GUI projects or to the interfaces of historical external fitting packages.

14 Stellar Population Libraries

14.1 Purpose and scope

The stellar-population-library layer is treated as a standalone scientific component of FitSpec rather than as an implementation detail of the fitting routine. This decision emerged from the library-development work itself: BPASS, Starburst99, SLUG2, and e-MILES differ not only in file format, but also in evolutionary assumptions, atmosphere assignments, metallicity conventions, age sampling, wavelength coverage, spectral resolution, and the distinction between stellar-only and nebular-processed products. The fitting code should therefore not be responsible for understanding the native distribution format of every synthesis package. Instead, each source library is translated once into a self-describing, fitting-oriented HDF5 representation, validated independently, and exposed through a common lazy-loading interface.

This architecture follows a strict separation of concerns:

1. **source-model layer:** preserve the original release products and their scientific meaning;
2. **library-builder layer:** translate source-specific files into a common HDF5 organization without changing the spectra scientifically;
3. **library-loader layer:** retrieve only the model family, age, metallicity, and wavelength range requested by the user;
4. **stellar-fitting layer:** apply observation-dependent transformations such as foreground extinction, internal attenuation, velocity shift, stellar velocity dispersion, instrumental line-spread-function matching, and resampling.

The guiding principle is that a permanent stellar library should contain the *native model information*, not an instrument-specific version of the model. This is particularly important because FitSpec is intended to operate on both ultraviolet and optical spectroscopy, and the same SSP may need to be compared against very different instrumental resolutions and wavelength grids.

The work summarized here used the public BPASS v2.2.1 release and its user manual (Eldridge et al., 2017b; Stanway & Eldridge, 2018b; BPASS collaboration, 2018), the Starburst99 software and documentation (Leitherer et al., 1999b; Vázquez & Leitherer, 2005; Leitherer et al., 2010, 2014; Starburst99 team, ???a,b,c), the SLUG2 code and documentation (da Silva et al., 2012; Krumholz et al., 2015b; SLUG developers, ???a,b,c,d,e), and the MILES/e-MILES model family accessed through MILESpy (Sánchez-Blázquez et al., 2006; Vazdekis et al., 2010, 2016b; MILESpy developers, ???a,b,c).

14.2 Common HDF5 design

The HDF5 files are designed for direct scientific use rather than simply archival compression. The common design requirements are:

- spectra remain in their native physical units whenever those units are supplied by the source model;
- wavelength arrays remain on the native model grid unless a source-specific reconstruction requires interpolation;
- ages and metallicities are stored explicitly and never inferred later from filenames alone;
- evolutionary tracks, rotation state, IMF choice, atmosphere choice, binary/single-star population choice, and other source-specific parameters are retained as HDF5 groups or attributes;
- the array orientation is standardized, preferentially as (`age`, `wavelength`);
- datasets are compressed and chunked so that one requested SSP or a limited wavelength interval can be read without loading the entire model library;
- observational processing is excluded from the build stage;
- source provenance is written into the output file.

This organization is especially important for the large libraries. The fitting routine must never load all stellar libraries into memory. It should open only the file selected by the user, and then only the requested branch within that file.

14.3 BPASS v2.2.1

14.3.1 Source release and motivation

BPASS is the binary-population branch of the stellar-library suite. Its distinguishing feature is the explicit treatment of binary stellar evolution in population synthesis (Eldridge et al., 2017b; Stanway & Eldridge, 2018b). The public BPASS site identifies v2.2.1 as the principal v2.2 population-synthesis release and provides both population products and individual stellar-model products (BPASS collaboration, 2018, 2026). The project used the locally downloaded v2.2.1 release rather than obtaining models dynamically from the web.

The release inventory included multiple IMF archives,

```

bpass_v2.2.1_imf_chab100.tar.gz
bpass_v2.2.1_imf_chab300.tar.gz
bpass_v2.2.1_imf100_100.tar.gz
bpass_v2.2.1_imf100_300.tar.gz
bpass_v2.2.1_imf135_100.tar.gz
bpass_v2.2.1_imf135_300.tar.gz
bpass_v2.2.1_imf135all_100.tar.gz
bpass_v2.2.1_imf170_100.tar.gz
bpass_v2.2.1_imf170_300.tar.gz

```

together with the individual stellar-model archive `bpass-v2.2-newmodels.tar.gz`, the BPASS v2.2.1 manual, and the WM-Basic atmosphere archive `v2.1_WMBasic_OB_models.tar`. The BPASS release documentation explicitly warns that the complete release is very large, particularly when individual stellar models are included (BPASS collaboration, 2018).

14.3.2 Population-spectrum products

Inspection of the released `spectra-bin-*` products confirmed the standard BPASS population-spectrum organization described in the manual: one wavelength column followed by population fluxes at discrete ages. For the inspected `spectra-bin-imf135_300.z020.dat.gz` file, the array contained 52 columns and 100,000 wavelength rows. The first column is wavelength and the remaining 51 columns are age-dependent population spectra. This structure is well suited to direct conversion into an HDF5 population library because the age dimension and wavelength dimension are already explicit.

The high-resolution `hrs-bin-*` products were also inspected. Their organization differs substantially from the standard `spectra-*` files and they were therefore not assumed to be interchangeable with the population SED products without validation. This distinction is retained in the documentation because file naming alone is insufficient evidence that two BPASS products represent the same spectral object.

14.3.3 Individual BPASS stellar models

The individual BPASS stellar-evolution files contained in `NEWSINMODES` and the binary-model archives were inspected because the original goal was not merely to consume precomputed population SEDs, but to understand whether the stellar library itself could be reconstructed. The BPASS manual documents the `sneplot` format and identifies the most important columns, including age, radius, effective temperature, luminosity, stellar mass, surface abundances, binary period and separation, companion properties, and IMF weights (BPASS collaboration, 2018). In particular,

$$\log g = \log g_{\odot} + \log_{10}(M/M_{\odot}) - 2 \log_{10}(R/R_{\odot}), \quad (51)$$

so the atmosphere-selection parameters can be derived from the evolutionary model. A direct check of a solar-metallicity $20 M_{\odot}$ model gave $\log g \simeq 4.01$, consistent with the dwarf WM-Basic atmosphere branch.

14.3.4 WM-Basic atmosphere archive

The BPASS release includes a set of WM-Basic O-star atmospheres originally released with BPASS v2.1. The BPASS manual states that these are direct WM-Basic outputs rather than the rebinned versions used internally by BPASS, and that the first two spectral columns correspond to wavelength

and flux while the later columns contain continuum-subtraction information. The WM-Basic atmosphere formalism is described by [Pauldrach et al. \(2001\)](#), and the theoretical UV library incorporated into Starburst99 is described by [Leitherer et al. \(2010\)](#).

The local archive contained 242 atmosphere files divided among:

- **dw**: dwarf atmospheres, approximately $\log g = 4.0$;
- **hg**: high-gravity atmospheres, approximately $\log g = 4.5$;
- **sg**: supergiant atmospheres, with temperature-dependent lower gravities.

The filenames encode atmosphere class, metallicity and effective temperature. The grid includes multiple metallicities and temperatures from roughly 25–50 kK. The atmosphere files do *not* share a universal wavelength grid. Even for a fixed gravity class and fixed T_{eff} , the number and locations of wavelength points can change with metallicity. Consequently, these spectra cannot safely be stacked by array index alone.

The direct WM-Basic output also has strongly non-uniform spectral sampling. In the far-UV, the inspected models contained hundreds of points between 1200 and 1800 Å, with typical spacings near 1 Å, while the optical sampling becomes much coarser. This is a property of the source atmosphere output, not a defect introduced by the HDF5 conversion.

14.3.5 Atmosphere normalization and reconstruction

The atmosphere grid is sufficient to reconstruct a spectral *shape* for a stellar evolutionary state after interpolation in T_{eff} , $\log g$, and metallicity, but the direct atmosphere flux normalization must be tied consistently to the luminosity and radius of the corresponding BPASS evolutionary model before the result can be interpreted as an absolute stellar luminosity. The BPASS track luminosity therefore remains the controlling normalization quantity. The reconstruction workflow must preserve bolometric consistency rather than treating the tabulated atmosphere amplitude as an arbitrary population weight.

This distinction was one of the central lessons of the BPASS exercise: matching atmosphere shape to an evolutionary point and assigning an absolute luminosity are separate operations.

14.3.6 Population-library product

After the source products and stellar/atmosphere structure were understood, the fitting-oriented population library was consolidated into

BPASS_v2.2.1_population_library.h5

rather than retaining the approximately multi-gigabyte directory tree as the fitting interface. The HDF5 conversion provides compression, one-file portability, explicit metadata, and lazy access to individual population branches.

The BPASS fitting interface must also preserve the distinction between intrinsic stellar spectra and nebular-processed products. The BPASS project explicitly distributes Cloudy-processed nebular products separately and cautions that nebular parameters must be chosen for the intended science ([BPASS collaboration, 2026](#)). The base fitting library should therefore be treated as a stellar library unless a separately identified nebular branch is loaded.

14.4 Starburst99 / pySB99

14.4.1 Scientific basis

Starburst99 is a population-synthesis code designed for the spectrophotometric modeling of star-forming systems (Leitherer et al., 1999b; Vázquez & Leitherer, 2005). The official Starburst99 site documents the evolution of the code, including the incorporation of line-blanketed O-star and WR atmospheres, theoretical UV libraries, a 0.3 Å high-resolution optical library, rotating Geneva tracks, and the Potsdam Wolf-Rayet library (Martins et al., 2005; Starburst99 team, ???a,b,d). The theoretical UV WM-Basic library spans 900–3000 Å at high sampling and is described in Leitherer et al. (2010); Starburst99 v7 added rotating Geneva tracks and PoWR-based UV spectra (Leitherer et al., 2014).

The atmosphere ingredients relevant to the Starburst99-style synthesis include the Lejeune/Kurucz-type grid for ordinary stars (Lejeune et al., 1997), WM-Basic/Pauldrach atmospheres for hot OB stars (Pauldrach et al., 2001), and line-blanketed WR atmospheres (Smith et al., 2002). The official Starburst99 chronology provides the historical mapping between these atmosphere updates and code releases (Starburst99 team, ???b).

14.4.2 pySB99 population-library construction

For FitSpec, the local Python Starburst99 implementation was used to generate an HDF5 population library rather than relying on interactive web runs. The completed build used:

- Geneva evolutionary tracks;
- non-rotating models in the initial library;
- a $10^6 M_{\odot}$ instantaneous burst normalization;
- a Kroupa-like broken-power-law IMF configuration used by the local pySB99 implementation;
- ages out to 50 Myr;
- two high-resolution spectral branches, labeled WM and PoWR;
- metallicity labels MWC, MW, LMC, SMC, IZw18, XMP, and Z0.

The final HDF5 build produced 14 successful groups:

- seven metallicity labels in the WM branch;
- seven metallicity labels in the PoWR branch.

Each group contains 500 age-dependent spectra.

The wavelength grids were measured directly from the generated HDF5:

- **WM:** 900.00–2998.45 Å, 4199 pixels;
- **PoWR:** 920.00–3100.00 Å, 10,000 pixels.

These are therefore UV libraries rather than optical fitting libraries.

The builder initially encountered non-monotonic wavelength output. That issue was fixed before the final HDF5 was accepted, illustrating the general validation rule that a successful synthesis run

is not sufficient: the wavelength axis itself must be checked before a model is admitted to the fitting library.

The local pySB99 README also generated caution messages at extreme metallicity labels where the high-resolution atmosphere coverage is limited. Those warnings are retained conceptually in the library design: a model can be numerically available without being equally reliable for precision line fitting at every metallicity.

14.4.3 Nebular continuum policy

The Starburst99/pySB99 HDF5 constructed here is treated as a stellar library. Starburst99 can produce stellar+nebular continua, and the official model tables explicitly distinguish stellar-only and stellar+nebular outputs (Starburst99 team, ???e). For FitSpec, nebular continuum should therefore be an explicit option rather than silently mixed into the base stellar templates. This mirrors the BPASS policy and prevents double counting if nebular continuum is modeled separately in the fitting pipeline.

14.5 SLUG2

14.5.1 Why SLUG was added

SLUG is designed to model stellar populations while explicitly allowing stochastic sampling of the IMF and other population distributions (da Silva et al., 2012; Krumholz et al., 2015b). The SLUG documentation emphasizes that it can generate both cluster and galaxy spectra and that its core distinction from traditional synthesis codes is its probabilistic treatment of the underlying stellar population (SLUG developers, ???a). This makes SLUG scientifically complementary to BPASS and Starburst99.

The initial FitSpec library, however, was constructed as a *deterministic* branch so that the evolutionary-track dependence could be explored independently of stochastic sampling. A stochastic branch can be added later without changing the basic HDF5 design.

14.5.2 Compilation and runtime environment

The local SLUG2 source was compiled on macOS/Apple Silicon. The build exposed several environment-level dependencies that should be documented for reproducibility. SLUG requires Boost and GSL, while CFITSIO is required for FITS support and for some track sets (SLUG developers, ???c). The first compilation attempt failed because `fitsio.h` was not found. After CFITSIO support was resolved, the executable built successfully, but macOS initially failed to locate the conda GSL dynamic library. Runtime execution required

```
export DYLD_LIBRARY_PATH="${CONDA_PREFIX}/lib:${DYLD_LIBRARY_PATH}"
```

in the active shell.

This is an environment configuration issue, not a scientific property of the models, but it is essential for reproducibility of the library build.

14.5.3 Track families discovered in the local SLUG installation

The local SLUG distribution provided:

- MIST v1.0, both `vvcr000` and `vvcr040`;

- PARSEC v1.2;
- Geneva 2013 at $Z = 0.014$ and $Z = 0.002$, both non-rotating and rotating at 40% of break-up.

The corresponding evolutionary model references are [Dotter \(2016\)](#); [Choi et al. \(2016\)](#), [Bressan et al. \(2012\)](#), [Ekström et al. \(2012\)](#), and [Georgy et al. \(2013\)](#). The SLUG parameter documentation independently lists these track families and their supported metallicity conventions ([SLUG developers, 2017b](#)).

14.5.4 Spectral synthesis mode and atmosphere assignment

The SLUG parser accepts the normal `sb99` spectral-synthesis mode and, in the local source tree, a special `sb99hruv` mode. The documented `sb99` mode assigns Pauldrach atmospheres to OB stars, Hillier-type WR atmospheres to Wolf-Rayet stars, and Kurucz/Lejeune-type atmospheres to other stars ([SLUG developers, 2017b](#); [Pauldrach et al., 2001](#); [Lejeune et al., 1997](#); [Smith et al., 2002](#)).

An important empirical result of the build was that `sb99hruv` repeatedly produced a segmentation fault for the tested Geneva configuration, whereas the same track configuration succeeded with ordinary `sb99`. The production library therefore uses `sb99`; `sb99hruv` is not enabled merely because the parser accepts the keyword. This is a deliberate validation-based restriction.

14.5.5 Nebular processing must be disabled explicitly

SLUG’s parameter documentation states that `compute_nebular` defaults to 1 ([SLUG developers, 2017b](#)). The first stellar-only build failed with

```
mismatch in continuum wavelength grid between nebular and stellar data
```

because nebular processing was still being invoked. The builder was corrected to write

```
compute_nebular 0
```

explicitly. This is now a mandatory part of the SLUG stellar-library build. The SLUG documentation distinguishes the intrinsic stellar L_λ from the nebular-processed spectrum and gives the output luminosity units as $\text{erg s}^{-1} \text{\AA}^{-1}$ ([SLUG developers, 2017a,d](#)).

14.5.6 Deterministic versus stochastic mode

SLUG’s `min_stoch_mass` controls the mass below which the IMF is treated continuously ([SLUG developers, 2017b](#)). During debugging this keyword was temporarily set to zero, which made nominal “deterministic” test groups single stochastic realizations. The production builder corrected this: deterministic mode sets the stochastic threshold above the IMF upper limit, while the optional stochastic mode sets it to zero and stores multiple realizations separately.

The HDF5 structure is therefore designed to support both:

```
/models/deterministic/...
/models/stochastic/mass_.../...
```

even though the completed first-generation library contains only the deterministic branch.

14.5.7 The PARSEC interpolation problem

The PARSEC debugging exercise is important enough to document because it exposed a subtle failure mode that could easily have produced an incomplete library.

When a PARSEC FITS file was passed directly through the generic `tracks=<filename>` interface, SLUG attempted to interpret it using the MIST file path and failed. The builder therefore switched to the named track set

```
tracks parsec_v1.2
```

with a separate solar-normalized metallicity parameter.

Inspection of `slug_tracks_parsec.cpp` showed that the PARSEC implementation adopts

$$Z_{\odot} = 0.014,$$

so the parameter passed to SLUG is

$$Z_{\text{input}} = Z_{\text{absolute}}/0.014.$$

A second problem then appeared: most metallicities failed while $Z = 0.0001$ and $Z = 0.014$ worked. Inspection of `slug_tracks_slime.cpp` showed why. If the requested metallicity does not compare as an *exact* match, SLUG enters its metallicity-interpolation path and reads all available PARSEC files into arrays with a common track-count dimension. The native PARSEC files do not have a common number of mass tracks. Direct inspection showed 98–105 tracks depending on metallicity. Each file was internally self-consistent, but the cross-metallicity interpolation path assumed a common array size.

The final failure was therefore triggered not by corrupt PARSEC files, but by floating-point serialization. Writing the solar-normalized metallicity with 15 significant digits sometimes failed SLUG’s exact-match comparison and sent execution into the problematic interpolation branch. Writing the value with 17 significant digits,

```
metallicity_line = f"metallicity {metallicity_zsun:.17g}"
```

preserved the round-trip double precision and forced exact native-grid selection. A five-metallicity test then succeeded, followed by the complete PARSEC build.

This implementation detail is recorded because future changes to the builder must not casually reduce the metallicity-string precision.

14.5.8 Completed SLUG HDF5 library

The final deterministic SLUG library contains 49 successful groups:

- **30 MIST groups:** 15 metallicities for each of `vvcr000` and `vvcr040`;
- **15 PARSEC v1.2 groups:** $Z = 0.0001, 0.0002, 0.0005, 0.001, 0.002, 0.004, 0.006, 0.008, 0.010, 0.014, 0.017, 0.02$
- **4 Geneva 2013 groups:** $Z = 0.002$ and 0.014 , each at 0 and 40% rotation.

All groups use:

- `specsyn_mode = sb99`;
- ages 1–50 Myr in 1 Myr steps;

- a nominal $10^6 M_{\odot}$ SSP normalization;
- stellar-only spectra;
- one common wavelength grid.

The HDF5 hierarchy is:

```
/models/deterministic
  /MIST
    /vvcrit000/sb99/feh_...
    /vvcrit040/sb99/feh_...
  /PARSEC_v1.2
    /native/sb99/Z...
  /GENEVA2013
    /vvcrit000/sb99/Z...
    /vvcrit040/sb99/Z...
```

14.5.9 SLUG wavelength coverage and scientific use

All 49 models share one 1221-pixel wavelength grid from 91 Å to 1.6×10^6 Å. The grid is broad but coarse:

Region	Range (Å)	N_{λ}	Median $\Delta\lambda$ (Å)	Median sampling R
FUV blue	900–1200	30	10	~ 105
COS FUV	1200–1800	63	10	~ 155
NUV	1800–3000	117	10	~ 235
Near-UV	3000–4000	51	20	~ 175
Optical	4000–7000	150	20	~ 275
Red optical	7000–10000	150	20	~ 425

The quoted R values describe grid sampling only, not a physical instrumental or line-spread-function resolution. The practical conclusion is that the SLUG **sb99** library is valuable for broad stellar SEDs and for testing evolutionary-track dependence, but it should not be treated as a precision stellar-line library. Artificially interpolating the 10–20 Å sampling onto a finer wavelength grid would not create new spectral information.

14.6 e-MILES

14.6.1 Why e-MILES is the optical backbone

The MILES empirical stellar library covers the optical at moderate resolution ([Sánchez-Blázquez et al., 2006](#)), and the MILES SSP models provide a widely used empirical-basis population-synthesis framework ([Vazdekis et al., 2010](#)). e-MILES extends the SSP wavelength coverage into the UV and infrared ([Vazdekis et al., 2016b](#)). MILESpy provides a modern Python interface to the MILES/e-MILES repositories and documents the available SSP families, IMFs, isochrones, and wavelength coverage ([MILESpy developers, ???a,c](#)).

The e-MILES branch therefore fills the principal gap left by the young-population UV libraries: high-information-content optical and near-IR SSP spectra at intermediate and old ages.

14.6.2 Repository acquisition and configuration

MILESpy v1.0 was installed locally. Its repository manager identifies the official SSP repository as

```
EMILES_SSP_v9.1.hdf5
```

and validates downloaded files with a stored MD5 hash. The repository file used for FitSpec is approximately 5.29 GiB.

MILESpy's default repository location is the package installation directory. The project instead stores the repository in the stellar-library workspace. MILESpy's own documentation notes that repository files are downloaded on first use and that the repository directory can be configured ([MILESpy developers, ???b](#)). The local configuration therefore sets

```
repository_folder = ../../stellar_libraries/MILES/
```

and then initializes the library using the canonical source name `EMILES_SSP`. Passing the absolute HDF5 filepath directly as `source` was not sufficient because the MILESpy LSF-selection logic expects a recognized source identifier.

14.6.3 Native repository structure

The source HDF5 contains global metadata arrays including age, metallicity, alpha abundance, IMF type, IMF slope, isochrone, stellar/remnant/gas/total masses, V -band mass-to-light quantities, M_V , filename, wavelength, and spectra. The spectral tree is organized as

```
/spec/P/<imf>/data
/spec/T/<imf>/data
```

where:

- P = Padova00;
- T = BaSTI.

The Padova00 evolutionary basis traces to the Padova stellar-evolution calculations ([Girardi et al., 2000](#); [Padova stellar-evolution group, ????](#)), while the BaSTI basis is described by [Pietrinferni et al. \(2004\)](#). The IMF options include Kroupa and Chabrier forms ([Kroupa, 2001](#); [Chabrier, 2003](#)), together with the MILES unimodal and bimodal parameterizations.

The readable branches were:

- ku: Kroupa Universal;
- kb: Kroupa Revised;
- ch: Chabrier;
- bi: bimodal.

The `un` group raised an HDF5 `wrong B-tree signature` error for both Padova00 and BaSTI when the source repository was traversed. Because the other branches loaded successfully through MILESpy and h5py, the issue appeared localized rather than a failure of the whole repository. The unimodal branch is therefore excluded from the fitting-oriented repack until independently validated.

14.6.4 Wavelength grid

The native e-MILES wavelength vector contains 53,689 points from 1680.2 to 49,999.4 Å with a constant 0.9 Å sampling. This is consistent with the extended wavelength coverage documented for e-MILES (Vazdekis et al., 2016b; MILESpy developers, ???a).

Measured grid sampling is:

Region	Range (Å)	Median $\Delta\lambda$ (Å)	Median sampling R
UV	1680–3000	0.9	~ 2600
Near-UV	3000–4000	0.9	~ 3900
Optical	4000–7000	0.9	~ 6100
Red optical	7000–10000	0.9	~ 9400
Near-IR	10000–25000	0.9	~ 19400
25000–50000	25000–50000	0.9	~ 41700

Again, these R values are sampling ratios, not the intrinsic spectral resolving power. The fitting routine must use the e-MILES line-spread function rather than infer the physical resolution from pixel spacing.

14.6.5 Padova00 grid

For Kroupa Universal and Padova00, the library contains a rectangular grid of 350 SSPs:

- 50 ages from 0.0631 to 17.7828 Gyr;
- seven metallicities:

$$[M/H] = [-2.32, -1.71, -1.31, -0.71, -0.40, 0.00, +0.22].$$

The $50 \times 7 = 350$ model count exactly matches the loaded MILESpy branch.

14.6.6 BaSTI grid

For Kroupa Universal and BaSTI, the library contains a rectangular grid of 636 SSPs:

- 53 ages from 0.03 to 14 Gyr;
- twelve metallicities:

$$[M/H] = [-2.27, -1.79, -1.49, -1.26, -0.96, -0.66, -0.35, -0.25, +0.06, +0.15, +0.26, +0.40].$$

The $53 \times 12 = 636$ model count again exactly matches the loaded branch. The youngest BaSTI SSP is 30 Myr, producing a scientifically useful 30–50 Myr overlap with the young-population libraries.

14.6.7 Fitting-oriented e-MILES HDF5

Although the MILESpy repository is already HDF5, its orientation and global metadata layout are optimized for the package rather than for the FitSpec loader. A dedicated builder,

```
build_EMILES_population_library.py
```

therefore creates

EMILES_population_library.h5

with a hierarchy organized by isochrone, IMF, IMF slope where relevant, metallicity, and age:

```

/wavelength_A
/models
  /Padova00
    /ku
      /mh_m2p32
        age_Gyr
        flux
        ...
    /kb
    /ch
    /bi
      /slope_...
      /mh_...
  /BaSTI
  ...

```

Each flux dataset is stored as (**age**, **wavelength**) and chunked approximately as (1, 2048). The builder copies aligned scalar metadata such as stellar and remnant mass, mass-to-light quantities, M_V , alpha abundance, and IMF slope. The spectra are not resampled, broadened, dereddened, or renormalized.

14.7 Cross-library roles

The four libraries are complementary rather than interchangeable:

Library	Primary regime	age	Spectral role	Main distinguishing physics / use
BPASS	young to extended		UV/stellar-population synthesis	Binary evolution; native BPASS population products and atmosphere-aware reconstruction
Starburst99	young		high-resolution UV	Massive-star UV spectra, WM-Basic/PoWR atmosphere branches, Geneva-based young SSPs
SLUG2	young		broad SED / population tests	MIST, PARSEC, Geneva track comparisons; stochastic capability; not a high-resolution line library in current sb99 mode
e-MILES	$\gtrsim 30$ Myr		optical/near-IR fitting backbone	Empirical MILES basis, Padova00/BaSTI SSPs, dense 0.9 Å sampling

The overlap between these libraries is scientifically useful. In particular, the 30–50 Myr overlap between BaSTI e-MILES and the young-population libraries can be used to test whether the fitting framework introduces discontinuities when the preferred model family changes with age.

14.8 Nebular continuum policy across libraries

Nebular continuum is not treated as an implicit component of the stellar templates. This is a deliberate design rule:

- BPASS distributes Cloudy-processed nebular products separately from its base stellar products (BPASS collaboration, 2026);
- Starburst99 distinguishes stellar-only and stellar+nebular continua in its standard outputs (Starburst99 team, ???e);
- SLUG defaults to computing nebular processing unless `compute_nebular=0` is set explicitly (SLUG developers, ???b);
- e-MILES is used as a stellar SSP library.

The fitter should therefore include nebular continuum only when explicitly requested. This prevents accidental double counting when emission-line or nebular-continuum components are modeled elsewhere.

14.9 Resolution and sampling policy

No library is upsampled merely to match an observed pixel grid. Model sampling and physical resolution are distinct quantities. The fitter must:

1. determine the intrinsic model resolution or LSF;
2. determine the observed instrumental LSF;
3. convolve the higher-resolution component to the lower-resolution target where mathematically valid;
4. only then resample onto the comparison grid.

A coarse source spectrum such as the current SLUG `sb99` grid cannot be converted into a precision line template by interpolation. Conversely, the 0.9 Å e-MILES pixel spacing must not be mistaken for its physical FWHM resolution.

14.10 Lazy access and compartmentalization

The user must be able to select a stellar library without paying the memory cost of all libraries. The eventual loader should therefore follow a pattern such as:

```
with StellarLibrary(path) as lib:
    models = lib.load(
        family="BaSTI",
        imf="ku",
        metallicity=[-0.35, 0.06],
        age_range=(0.03, 1.0),
        wavelength_range=(3600.0, 7000.0),
    )
```

Only the requested HDF5 datasets should be materialized. This rule is especially important for e-MILES and BPASS, whose source repositories are several gigabytes.

14.11 Validation checklist

Every builder must perform, or be followed by, a branch-level validation pass. At minimum:

- confirm that wavelength vectors are finite and strictly increasing;
- verify that flux-array dimensions match the stored age and wavelength axes;
- verify that every age/metallicity label maps to the intended source spectrum;
- check for duplicate ages and duplicate wavelength points;
- verify source and output wavelength ranges;
- record native sampling and, separately, physical spectral resolution where known;
- verify that no branch is silently unreadable;
- confirm that the source spectra are unchanged except for array orientation/storage layout;
- test lazy loading of one SSP and one wavelength interval;
- verify that nebular processing is off unless the branch is explicitly identified as nebular;
- retain source warnings concerning atmosphere coverage or metallicity extrapolation.

The work described above demonstrates why this is necessary. Examples include non-monotonic wavelength output in the initial pySB99 build, SLUG’s default nebular processing, the **sb99hruv** segmentation fault, the PARSEC exact-metallicity/interpolation failure, and the unreadable e-MILES unimodal HDF5 branch. None of these problems would have been caught by checking only that an output file existed.

14.12 Builder scripts and permanent products

The current library-construction products are:

Library	Builder / preparation layer	Permanent fitting file
BPASS	BPASS population reconstruction and HDF5 conversion	BPASS_v2.2.1_population_library.h5
Starburst99	build_SB99_population_library.py	SB99_population_library.h5
SLUG2	build_SLUG_population_library.py	SLUG2_population_library.h5
e-MILES	build_EMILES_population_library.py	EMILES_population_library.h5

The original source repositories should be retained separately for reproducibility, but the fitting code should interact with the permanent HDF5 products through source-specific lazy loaders.

14.13 External documentation and web resources used

The implementation was informed not only by the peer-reviewed model papers but also by the official software documentation. The most important web resources consulted were:

- the BPASS v2.2.1 release page and current BPASS project page ([BPASS collaboration, 2018, 2026](#));

- the Starburst99 home page, parameter interface, chronology, release/news page, and original model-table index ([Starburst99 team, ???a,c,b,d,e](#));
- the SLUG introduction, compilation guide, parameter specification, output-format documentation, and test-suite documentation ([SLUG developers, ???a,c,b,d,e](#));
- the MILESpy main documentation, installation instructions, and SSP tutorial ([MILESpy developers, ???a,b,c](#));
- the Padova evolutionary-track archive used to document the Padova00 provenance ([Padova stellar-evolution group, ???](#)).

These web citations are retained deliberately. They document implementation details—default keyword values, supported track names, repository behavior, output units, and software-release capabilities—that are not always stated in the corresponding journal papers.

14.14 References that should accompany use of each library

For publications or public releases that use these templates, the minimum scientific citations should follow the model ingredients actually selected:

- **BPASS v2.2.1:** [Eldridge et al. \(2017b\)](#) and [Stanway & Eldridge \(2018b\)](#);
- **Starburst99:** [Leitherer et al. \(1999b\)](#), with [Vázquez & Leitherer \(2005\)](#), [Leitherer et al. \(2010\)](#), and/or [Leitherer et al. \(2014\)](#) as appropriate to the selected tracks and atmosphere library;
- **SLUG:** [da Silva et al. \(2012\)](#) and [Krumholz et al. \(2015b\)](#), plus the selected evolutionary-track reference: MIST ([Dotter, 2016](#); [Choi et al., 2016](#)), PARSEC ([Bressan et al., 2012](#)), or Geneva ([Ekström et al., 2012](#); [Georgy et al., 2013](#));
- **e-MILES:** [Vazdekis et al. \(2016b\)](#), with the MILES foundation ([Sánchez-Blázquez et al., 2006](#); [Vazdekis et al., 2010](#)) and the selected isochrone/IMF references where appropriate ([Girardi et al., 2000](#); [Pietrinferni et al., 2004](#); [Kroupa, 2001](#); [Chabrier, 2003](#)).

References

- Arav, N., Kaastra, J., Kriss, G. A., et al. 2005, *The Astrophysical Journal*, 620, 665, doi: [10.1086/427088](#)
- Barlow, T. A., & Sargent, W. L. W. 1997, *The Astronomical Journal*, 113, 136, doi: [10.1086/118239](#)
- BPASS collaboration. 2018, BPASS v2.2.1 data release, <https://bpass.auckland.ac.nz/9.html>
- . 2026, Binary Population and Spectral Synthesis (BPASS), <https://warwick.ac.uk/fac/sci/physics/research/astro/research/catalogues/bpass/>
- Bressan, A., Marigo, P., Girardi, L., et al. 2012, *Monthly Notices of the Royal Astronomical Society*, 427, 127, doi: [10.1111/j.1365-2966.2012.21948.x](#)
- Calzetti, D., Armus, L., Bohlin, R. C., et al. 2000, *The Astrophysical Journal*, 533, 682, doi: [10.1086/308692](#)

- Carswell, R. F., & Webb, J. K. 2014, VPFIT: Voigt profile fitting program, Astrophysics Source Code Library, record ascl:1408.015
- Chabrier, G. 2003, Publications of the Astronomical Society of the Pacific, 115, 763, doi: [10.1086/376392](https://doi.org/10.1086/376392)
- Choi, J., Dotter, A., Conroy, C., et al. 2016, The Astrophysical Journal, 823, 102, doi: [10.3847/0004-637X/823/2/102](https://doi.org/10.3847/0004-637X/823/2/102)
- Cleveland, W. S. 1979, Journal of the American Statistical Association, 74, 829, doi: [10.1080/01621459.1979.10481038](https://doi.org/10.1080/01621459.1979.10481038)
- da Silva, R. L., Fumagalli, M., & Krumholz, M. R. 2012, The Astrophysical Journal, 745, 145, doi: [10.1088/0004-637X/745/2/145](https://doi.org/10.1088/0004-637X/745/2/145)
- Dotter, A. 2016, The Astrophysical Journal Supplement Series, 222, 8, doi: [10.3847/0067-0049/222/1/8](https://doi.org/10.3847/0067-0049/222/1/8)
- Ekström, S., Georgy, C., Eggenberger, P., et al. 2012, Astronomy & Astrophysics, 537, A146, doi: [10.1051/0004-6361/201117751](https://doi.org/10.1051/0004-6361/201117751)
- Eldridge, J. J., Stanway, E. R., Xiao, L., et al. 2017a, Publications of the Astronomical Society of Australia, 34, e058, doi: [10.1017/pasa.2017.51](https://doi.org/10.1017/pasa.2017.51)
- . 2017b, Publications of the Astronomical Society of Australia, 34, e058, doi: [10.1017/pasa.2017.51](https://doi.org/10.1017/pasa.2017.51)
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. 1996, in Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, 226–231
- Ganguly, R., Bond, N. A., Charlton, J. C., et al. 2003, The Astrophysical Journal, 598, 922, doi: [10.1086/379000](https://doi.org/10.1086/379000)
- Georgy, C., Ekström, S., Eggenberger, P., et al. 2013, Astronomy & Astrophysics, 558, A103, doi: [10.1051/0004-6361/201322178](https://doi.org/10.1051/0004-6361/201322178)
- Girardi, L., Bressan, A., Bertelli, G., & Chiosi, C. 2000, Astronomy and Astrophysics Supplement Series, 141, 371, doi: [10.1051/aas:2000126](https://doi.org/10.1051/aas:2000126)
- Golub, G. H., & Pereyra, V. 1973, SIAM Journal on Numerical Analysis, 10, 413, doi: [10.1137/0710036](https://doi.org/10.1137/0710036)
- Kramida, A., Ralchenko, Y., Reader, J., & NIST ASD Team. 2023, NIST Atomic Spectra Database (version 5.11), <https://physics.nist.gov/asd>
- Kroupa, P. 2001, Monthly Notices of the Royal Astronomical Society, 322, 231, doi: [10.1046/j.1365-8711.2001.04022.x](https://doi.org/10.1046/j.1365-8711.2001.04022.x)
- Krumholz, M. R., Fumagalli, M., da Silva, R. L., Rendaahl, T., & Parra, J. 2015a, Monthly Notices of the Royal Astronomical Society, 452, 1447, doi: [10.1093/mnras/stv1374](https://doi.org/10.1093/mnras/stv1374)
- . 2015b, Monthly Notices of the Royal Astronomical Society, 452, 1447, doi: [10.1093/mnras/stv1374](https://doi.org/10.1093/mnras/stv1374)

- Lawson, C. L., & Hanson, R. J. 1995, Solving Least Squares Problems (Philadelphia: Society for Industrial and Applied Mathematics), doi: [10.1137/1.9781611971217](https://doi.org/10.1137/1.9781611971217)
- Leitherer, C., Ekström, S., Meynet, G., et al. 2014, The Astrophysical Journal Supplement Series, 212, 14, doi: [10.1088/0067-0049/212/1/14](https://doi.org/10.1088/0067-0049/212/1/14)
- Leitherer, C., Ortiz Otálvaro, P. A., Bresolin, F., et al. 2010, The Astrophysical Journal Supplement Series, 189, 309, doi: [10.1088/0067-0049/189/2/309](https://doi.org/10.1088/0067-0049/189/2/309)
- Leitherer, C., Schaerer, D., Goldader, J. D., et al. 1999a, The Astrophysical Journal Supplement Series, 123, 3, doi: [10.1086/313233](https://doi.org/10.1086/313233)
- . 1999b, The Astrophysical Journal Supplement Series, 123, 3, doi: [10.1086/313233](https://doi.org/10.1086/313233)
- Lejeune, T., Cuisinier, F., & Buser, R. 1997, Astronomy and Astrophysics Supplement Series, 125, 229, doi: [10.1051/aas:1997373](https://doi.org/10.1051/aas:1997373)
- Martins, L. P., González Delgado, R. M., Leitherer, C., Cerviño, M., & Hauschildt, P. H. 2005, Monthly Notices of the Royal Astronomical Society, 358, 49, doi: [10.1111/j.1365-2966.2005.08703.x](https://doi.org/10.1111/j.1365-2966.2005.08703.x)
- MILESpy developers. ???a, MILESpy documentation, <https://miles-iac.github.io/milespy/>
- . ???b, MILESpy Installation, <https://miles-iac.github.io/milespy/installation.html>
- . ???c, SSP Models Tutorial, https://miles-iac.github.io/milespy/tutorials/example_ssp.html
- Morton, D. C. 2003, The Astrophysical Journal Supplement Series, 149, 205, doi: [10.1086/377639](https://doi.org/10.1086/377639)
- O’Leary, D. P., & Rust, B. W. 2013, Computational Optimization and Applications, 54, 579, doi: [10.1007/s10589-012-9492-9](https://doi.org/10.1007/s10589-012-9492-9)
- Osterbrock, D. E., & Ferland, G. J. 2006, Astrophysics of Gaseous Nebulae and Active Galactic Nuclei, 2nd edn. (Sausalito, CA: University Science Books)
- Padova stellar-evolution group. ???, Padova Stellar Evolutionary Tracks and Isochrones, <https://pleiadi.oapd.inaf.it/>
- Pauldrach, A. W. A., Hoffmann, T. L., & Lennon, M. 2001, Astronomy & Astrophysics, 375, 161, doi: [10.1051/0004-6361:20010805](https://doi.org/10.1051/0004-6361:20010805)
- Pietrinferni, A., Cassisi, S., Salaris, M., & Castelli, F. 2004, The Astrophysical Journal, 612, 168, doi: [10.1086/422498](https://doi.org/10.1086/422498)
- Sánchez-Blázquez, P., Peletier, R. F., Jiménez-Vicente, J., et al. 2006, Monthly Notices of the Royal Astronomical Society, 371, 703, doi: [10.1111/j.1365-2966.2006.10699.x](https://doi.org/10.1111/j.1365-2966.2006.10699.x)
- Savitzky, A., & Golay, M. J. E. 1964, Analytical Chemistry, 36, 1627, doi: [10.1021/ac60214a047](https://doi.org/10.1021/ac60214a047)
- Schaye, J., Carswell, R. F., & Kim, T.-S. 2007, Monthly Notices of the Royal Astronomical Society, 379, 1169, doi: [10.1111/j.1365-2966.2007.11862.x](https://doi.org/10.1111/j.1365-2966.2007.11862.x)
- Schwarz, G. 1978, The Annals of Statistics, 6, 461, doi: [10.1214/aos/1176344136](https://doi.org/10.1214/aos/1176344136)

- SLUG developers. ???a, Introduction to SLUG, <https://slug2.readthedocs.io/en/latest/intro.html>
- . ???b, Parameter Specification, <https://slug2.readthedocs.io/en/latest/parameters.html>
- . ???c, Compiling and Installing SLUG, <https://slug2.readthedocs.io/en/latest/compiling.html>
- . ???d, Output Files and Format, <https://slug2.readthedocs.io/en/latest/output.html>
- . ???e, Test Problems, <https://slug2.readthedocs.io/en/latest/tests.html>
- Smith, L. J., Norris, R. P. F., & Crowther, P. A. 2002, Monthly Notices of the Royal Astronomical Society, 337, 1309, doi: [10.1046/j.1365-8711.2002.06042.x](https://doi.org/10.1046/j.1365-8711.2002.06042.x)
- Stanway, E. R., & Eldridge, J. J. 2018a, Monthly Notices of the Royal Astronomical Society, 479, 75, doi: [10.1093/mnras/sty1353](https://doi.org/10.1093/mnras/sty1353)
- . 2018b, Monthly Notices of the Royal Astronomical Society, 479, 75, doi: [10.1093/mnras/sty1353](https://doi.org/10.1093/mnras/sty1353)
- Starburst99 team. ???a, Starburst99 Home Page, <https://massivestars.stsci.edu/starburst99/docs/default.htm>
- . ???b, Starburst99 Chronology, <https://massivestars.stsci.edu/starburst99/docs/history.html>
- . ???c, Starburst99 Simulation Parameters, <https://massivestars.stsci.edu/starburst99/docs/parameters.html>
- . ???d, What’s New in Starburst99, <https://massivestars.stsci.edu/starburst99/docs/new.html>
- . ???e, Navigate Through the Original Starburst99 Dataset, <https://massivestars.stsci.edu/starburst99/docs/table-index.html>
- Stark, P. B., & Parker, R. L. 1995, Computational Statistics, 10, 129
- Storey, P. J., & Zeippen, C. J. 2000, Monthly Notices of the Royal Astronomical Society, 312, 813, doi: [10.1046/j.1365-8711.2000.03184.x](https://doi.org/10.1046/j.1365-8711.2000.03184.x)
- Tepper-García, T. 2006, Monthly Notices of the Royal Astronomical Society, 369, 2025, doi: [10.1111/j.1365-2966.2006.10450.x](https://doi.org/10.1111/j.1365-2966.2006.10450.x)
- Vazdekis, A., Koleva, M., Ricciardelli, E., Röck, B., & Falcón-Barroso, J. 2016a, Monthly Notices of the Royal Astronomical Society, 463, 3409, doi: [10.1093/mnras/stw2231](https://doi.org/10.1093/mnras/stw2231)
- . 2016b, Monthly Notices of the Royal Astronomical Society, 463, 3409, doi: [10.1093/mnras/stw2231](https://doi.org/10.1093/mnras/stw2231)
- Vazdekis, A., Sánchez-Blázquez, P., Falcón-Barroso, J., et al. 2010, Monthly Notices of the Royal Astronomical Society, 404, 1639, doi: [10.1111/j.1365-2966.2010.16407.x](https://doi.org/10.1111/j.1365-2966.2010.16407.x)
- Vázquez, G. A., & Leitherer, C. 2005, The Astrophysical Journal, 621, 695, doi: [10.1086/427866](https://doi.org/10.1086/427866)
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, Nature Methods, 17, 261, doi: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)